

POLITECHNIKA ŚWIĘTOKRZYSKA Wydział Elektrotechniki, Automatyki i Informatyki	
Kierunek: Informatyka	Projekt: Symulacja komputerowa
Grupa: 2ID21A	Temat projektu: Symulacja restauracji z kelnerami
Członkowie zespołu: Daniel Cieślak e-mail: s092691@student.tu.kielce.pl	

SPIS TREŚCI

1. WPROWADZENIE	3
1.1. Definicja symulacji komputerowej	3
1.2. Problem restauracji w kontekście symulacji	3
1.3. Zarządzanie czasem oczekiwania na jedzenie	4
2. INSTRUKCJA URUCHOMIENIA APLIKACJI	5
3. OPIS PROJEKTU	9
3.1. Użyte technologie	9
3.2. Architektura systemu	10
3.3. Użyte rozkłady losowe	12
3.4. Ekrany aplikacji	14
3.5. Opis plików konfiguracyjnych	15
3.5.1. Plik CMakeLists.txt	15
3.5.2. Plik Installer.nsi	16
3.5.3. Plik Installer.nsi	18
3.5.4. Plik Installer.nsi	18
3.6. Opis najważniejszych fragmentów kodu	19
3.6.1. Plik Game.cpp	19
3.6.2. Plik MainMenu.cpp	22
3.6.3. Plik Level.cpp	32

3.6.4. Plik CharactersManager.cpp	40
4. WYNIKI SYMULACJI	3
4.1. Badanie wpływu liczby pracowników na efektywność funkcjonowania restauracji	52
4.2. Badanie wpływu liczby stolików na efektywność funkcjonowania restauracji	58
5. PODSUMOWANIE	62

1. WPROWADZENIE

1.1. Definicja symulacji komputerowej

Symulacja komputerowa to metoda badawcza i analityczna, w której model rzeczywistego systemu lub procesu jest odwzorowywany w środowisku komputerowym w celu analizy jego zachowania. Symulacje pozwalają na przewidywanie skutków zmian parametrów, testowanie różnych scenariuszy i podejmowanie decyzji bez potrzeby ingerencji w rzeczywisty system. W informatyce symulacje komputerowe znajdują zastosowanie w wielu dziedzinach, od logistyki i produkcji, poprzez zarządzanie ruchem miejskim, aż po gry i animacje interaktywne. Dzięki odpowiedniemu modelowaniu, możliwe jest uwzględnienie losowości zachowań elementów systemu, a także monitorowanie statystyk i efektów decyzji w czasie rzeczywistym.

1.2. Problem restauracji w kontekście symulacji

Jednym z interesujących problemów, który można badać za pomocą symulacji komputerowej, jest funkcjonowanie restauracji. Restauracja to system dynamiczny, w którym wiele podsystemów (klienci, kelnerzy, kucharze, stoliki) współdziała w czasie rzeczywistym. Zachowanie systemu zależy zarówno od liczby i konfiguracji zasobów (np. liczby kelnerów i szefów kuchni, ilości stolików), jak i od losowych zdarzeń (np. przybycie klientów, czas oczekiwania, rezygnacja z obsługi).

Symulacja pozwala na modelowanie:

- napływu klientów do restauracji oraz kolejek oczekujących na stół,
- przydzielania stolików i zasady ich zajmowania, w tym przypadki, gdy klient decyduje się zająć stół tylko dla siebie,
- pracy personelu, w tym kelnerów wykonujących różne zadania (wpuszczanie klientów, przyjmowanie i dostarczanie zamówień, zbieranie brudnych naczyń) oraz kucharzy przygotowujących dania,
- dynamiki procesu zamawiania i spożywania posiłków, a także losowości w rezygnacji z oczekiwania lub długości poszczególnych czynności.

Taka symulacja umożliwia obserwację wpływu różnych parametrów na wydajność restauracji i satysfakcję klientów, a także gromadzenie szczegółowych statystyk dotyczących przebiegu symulacji.

1.3. Cel i zakres projektu

Celem projektu było stworzenie aplikacji komputerowej do symulacji działania restauracji z wykorzystaniem języka C++ i biblioteki graficznej SFML. W ramach projektu opracowano:

- interaktywny interfejs użytkownika z możliwością konfiguracji parametrów symulacji, takich jak liczba kelnerów i kucharzy, liczba stolików dwu- i czteroosobowych, czas trwania symulacji oraz prędkość działania symulacji,
- model zachowania klientów, w tym przybywanie do restauracji, ustawianie się w kolejce, przydzielanie stolików, możliwość zajęcia stolika tylko dla siebie oraz rezygnację z oczekiwania, jeśli kolejka jest zbyt długa,
- model pracy personelu, w tym kelnerów wykonujących różne zadania (wpuszczanie klientów, przyjmowanie i dostarczanie zamówień, zbieranie brudnych naczyń) oraz kucharzy przygotowujących posiłki według przypisanych zleceń,
- system kolejek i przydzielania zadań, w którym kelnerzy obsługują klientów, przydzielają zlecenia do kuchni, dostarczają gotowe dania i zbierają brudne naczynia, a kucharze przygotowują posiłki według zamówień,
- monitorowanie i wizualizację przebiegu symulacji, w tym animacje klientów, kelnerów i kucharzy, wizualizację zajętości stolików, czas pozostały do końca symulacji oraz statystyki podsumowujące jej przebieg. Wszystkie zachowania postaci są widoczne w atrakcyjnej grafice 2D w stylu pixelart, co pozwala na intuicyjne obserwowanie działania systemu w czasie rzeczywistym,
- zapis i prezentację statystyk po zakończeniu symulacji, obejmujących m.in. liczbę klientów, rezygnacje, zamówienia i posiłki spożyte. Statystyki są wyświetlane na ekranie oraz zapisywane do pliku stats.txt wraz z datą i godziną przeprowadzonej symulacji,
- mechanizm zapisu i wczytywania ustawień, pozwalający na zachowanie parametrów z poprzednich uruchomień w pliku konfiguracyjnym .ini.

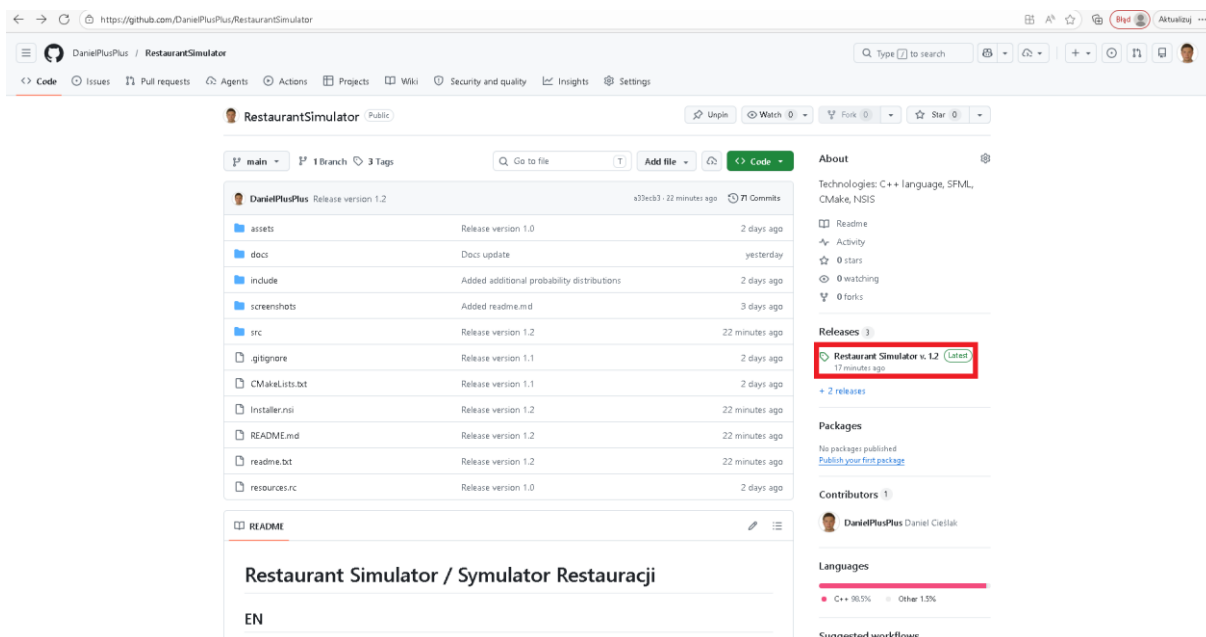
W efekcie powstała aplikacja umożliwia zarówno wizualną obserwację działania restauracji, jak i analizę ilościową różnych aspektów funkcjonowania systemu w warunkach ograniczonych zasobów i zmieniających się warunków, takich jak zmienna liczba klientów, długość kolejek czy czas oczekiwania na obsługę.

2. INSTRUKCJA URUCHOMIENIA APLIKACJI

Aplikacja „Restaurant Simulator” jest aktualnie dostępna wyłącznie na systemy operacyjne z rodziny Windows. Poniżej znajduje się lista kroków niezbędnych do pobrania i poprawnego uruchomienia programu:

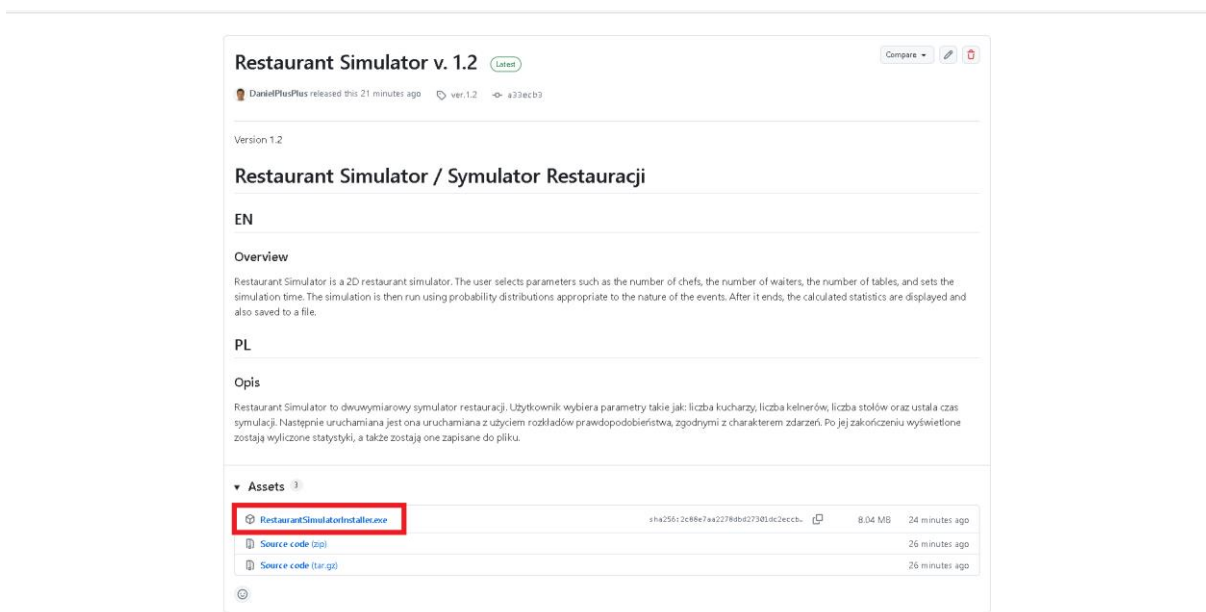
1. Wejście na stronę <https://github.com/DanielPlusPlus/RestaurantSimulator>

2. Wybranie pierwszej pozycji z panelu Releases



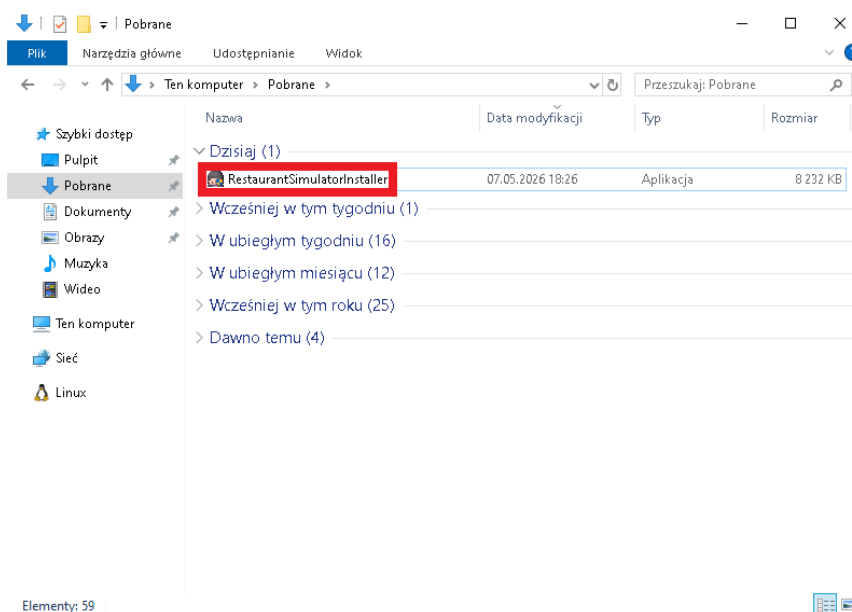
Rysunek 2.1. Repozytorium projektu

3. Wybranie pozycji RestaurantSimulatorInstaller.exe



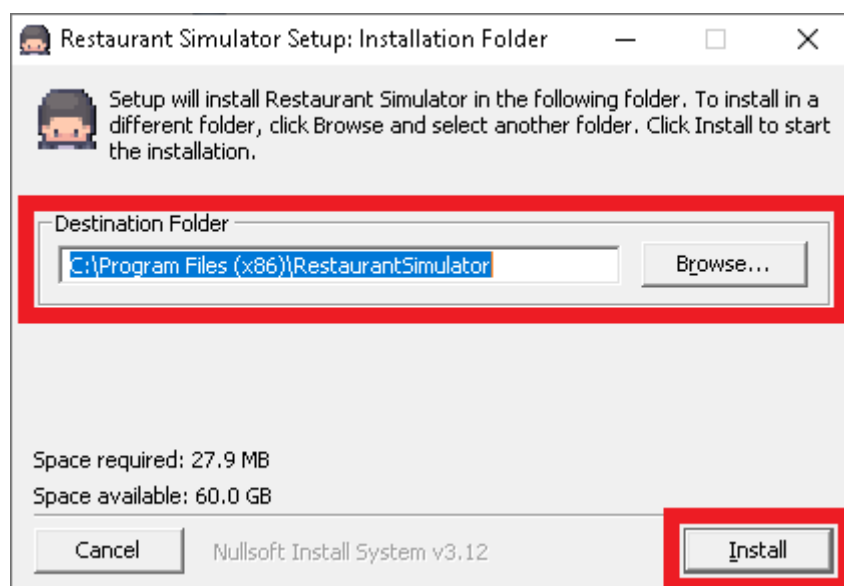
Rysunek 2.2. Strona z instalatorem aplikacji

4. Kliknięcie dwa razy lewym klawiszem myszy, w celu uruchomienia instalatora



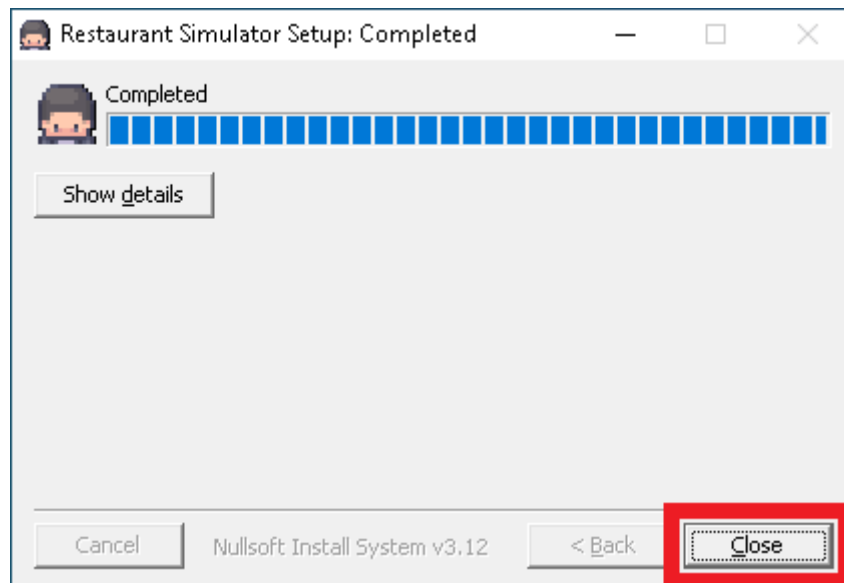
Rysunek 2.3. Pobrany instalator

5. Wybranie ścieżki instalacyjnej oraz kliknięcie przycisku Install



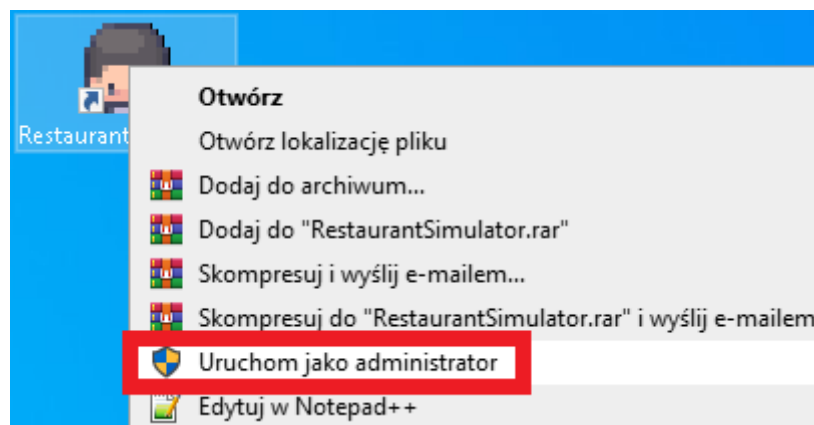
Rysunek 2.4. Uruchomiony instalator

6. Po zakończonej instalacji kliknięcie przycisku Close



Rysunek 2.5. Okno instalatora z zakończoną instalacją

7. Uruchomienie zainstalowanej aplikacji jako administrator



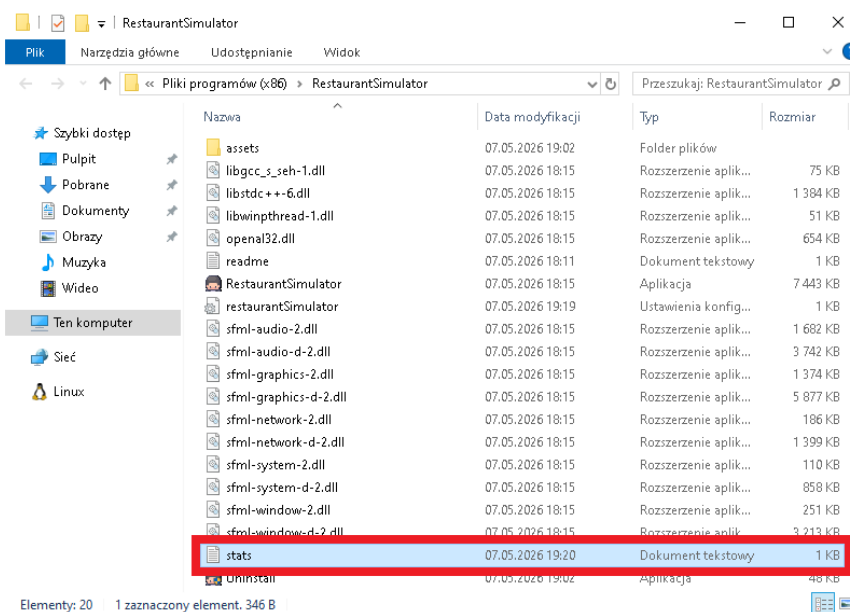
Rysunek 2.6. Menu kontekstowe po kliknięciu prawym przyciskiem myszy na skrót do aplikacji

8. Aplikacja jest uruchomiona. Aby zwiększyć wartość jakiegoś parametru lub wybrać opcję, należy kliknąć lewy przycisk myszy (LPM), a aby zmniejszyć wartość, kliknąć prawy przycisk myszy (PPM)



Rysunek 2.7. Uruchomiona aplikacja

9. Plik ze wszystkimi wynikami symulacji znajduje się w folderze, w którym została zainstalowana aplikacja



Rysunek 2.8. Plik stats.txt w folderze instalacyjnym aplikacji

3. OPIS PROJEKTU

3.1. Użyte technologie

Projekt Restaurant Simulator został zrealizowany z wykorzystaniem nowoczesnych narzędzi i bibliotek, które wspierają zarówno logikę gry, jak i warstwę wizualną oraz proces budowy i dystrybucji aplikacji. Są to:

1. C++

Głównym językiem programowania jest C++17, który zapewnia:

- wysoką wydajność potrzebną do symulacji w czasie rzeczywistym,
- możliwość zarządzania pamięcią i optymalizacji algorytmów,
- wsparcie dla programowania obiektowego i modularnej architektury aplikacji,
- bogaty ekosystem bibliotek i narzędzi kompatybilnych z CMake i SFML.

Kod źródłowy jest podzielony na nagłówki (.hpp) i implementacje (.cpp), co ułatwia separację odpowiedzialności i umożliwia łatwe testowanie komponentów.

2. SFML 2.6.2

Do obsługi grafiki, dźwięku i interfejsu użytkownika wykorzystano SFML (Simple and Fast Multimedia Library) w wersji 2.6.2. Biblioteka ta zapewnia:

- renderowanie 2D w czasie rzeczywistym, w tym animacje postaci w stylu pixelart,
- obsługę okien, zdarzeń wejścia (klawiatura, mysz) i dźwięku,
- łatwą integrację z C++ i systemami build w CMake,
- mechanizmy wsparcia dla skalowania okna, prędkości symulacji i dynamicznego rysowania scen.

SFML została dołączona do projektu w katalogu external/SFML-2.6.2 i podłączona w CMake poprzez find_package().

3. CMake

Projekt jest konfigurowany i kompilowany przy użyciu CMake – popularnego systemu automatyzacji budowy projektów C++:

- definiuje minimalną wymaganą wersję CMake (3.16) oraz standard C++17,
- automatycznie wyszukuje wszystkie pliki źródłowe w katalogu src/,
- konfiguruje i linkuje bibliotekę SFML,
- umożliwia kopiowanie zasobów, plików DLL i assetów do katalogu wykonywalnego po zakończeniu budowy.

Dzięki CMake projekt jest przenośny i łatwy do kompilacji zarówno na systemach Windows, jak i potencjalnie innych platformach.

4. NSIS (Nullsoft Scriptable Install System)

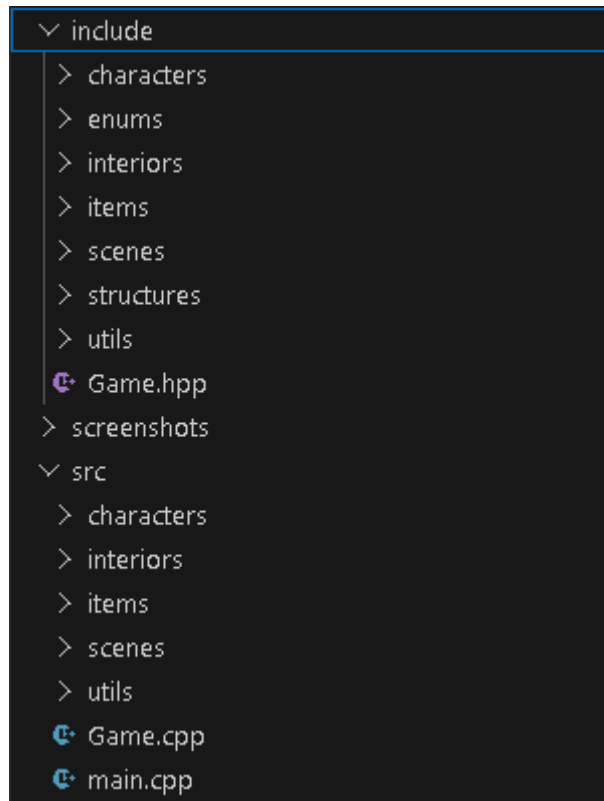
Do tworzenia instalatora dla systemu Windows wykorzystano NSIS. Skrypt instalatora definiuje m.in.:

- nazwę programu, ikonę oraz katalog instalacyjny (\$PROGRAMFILES\RestaurantSimulator),
- kopiowanie pliku wykonywalnego, bibliotek DLL, zasobów i dokumentacji (readme.txt),
- tworzenie skrótów w menu Start oraz na pulpicie,
- wpisy w rejestrze Windows umożliwiające odinstalowanie aplikacji,
- sekcję deinstalacyjną, która usuwa wszystkie pliki, katalogi i wpisy rejestru związane z programem.

Dzięki NSIS użytkownik otrzymuje wygodny instalator, który automatycznie instaluje aplikację i jej zasoby w systemie Windows.

3.2. Architektura systemu

Architektura systemu symulatora restauracji została zaprojektowana w sposób modułowy i warstwowy, a jego struktura plików i katalogów odzwierciedla podział funkcjonalny aplikacji. Główne elementy systemu znajdują się w katalogach include/ i src/, przy czym nagłówki klas i struktur umieszczono w include/, a implementacje w src/. Projekt wykorzystuje język C++ oraz bibliotekę SFML do renderowania grafiki 2D, a do budowania i konfiguracji projektu zastosowano CMake.



Rysunek 3.1. Struktura wykonanego projektu

Opis poszczególnych elementów struktury projektu:

1. Warstwa zarządzania grą (*Game* i *scenes*)

- *Game* – główna klasa aplikacji, odpowiedzialna za inicjalizację, pętlę gry, obsługę wejścia i przełączanie scen,
- *Scene*, *MainMenu*, *Level*, *Summary* – interfejs scen i ich implementacje, delegujące zadania do menedżerów postaci, stołów i potraw.

2. Postacie (*characters*)

- *Character* – klasa bazowa,
- *Waiter*, *Customer*, *Chef* – konkretne typy postaci z maszynami stanów,
- *CharactersManager* – menedżer postaci, odpowiedzialny za tworzenie, aktualizację, renderowanie i interakcje.

3. Definicje stanów (*enums*)

Typy wyliczeniowe definiujące stany postaci (kelner, kucharz, klient), kierunki ruchu oraz identyfikatory scen, umożliwiające implementację maszyn stanów i zarządzanie logiką postaci.

4. Stoły i wnętrza restauracji (*interiors*)

- *Table*, *TwoChairsTable*, *FourChairsTable* – reprezentacja stolików,

- TablesManager – menedżer stołów, odpowiedzialny za koordynację zajmowania i zwalniania krzeseł i stolików.
5. Zarządzanie potrawami (*items*)
- Dish – reprezentacja dania,
 - DishesManager – menedżer dań, odpowiedzialny za obsługę zamówień, przechowywanie dań i zarządzanie ich stanem
6. Struktury danych (*structures*)
- Zawierają współrzędne, pozycje stołów, krzeseł, potraw oraz ścieżki do tekstur. Wspomagają logikę gry i renderowanie obiektów w przestrzeni 2D.

3.3. Użyte rozkłady losowe

W projekcie symulacji restauracji wykorzystano kilka różnych rozkładów losowych, aby realistycznie modelować zachowania klientów i pracowników. Każdy rozkład został dobrany tak, aby odpowiadał rzeczywistym procesom występującym w restauracji, takim jak przybycie klientów, czas oczekiwania, decyzje o rezygnacji czy przebieg przygotowania i serwowania posiłków. Użyte rozkłady to:

1. Rozkład wykładniczy (`std::exponential_distribution`) – przyjscia klientów

Rozkład wykładniczy stosowany jest do modelowania czasów między przybyciami klientów, ponieważ napływ klientów jest procesem bezpamięciowym – prawdopodobieństwo przyjscia kolejnego klienta nie zależy od tego, kiedy pojawił się poprzedni. Dzięki temu rozkład wykładniczy dobrze oddaje naturalny, losowy rytm napływu gości do restauracji.

2. Rozkład normalny (`std::normal_distribution`) – czas do rezygnacji z kolejki

Czas, jaki klient jest skłonny poczekać w kolejce, modelowany jest rozkładem normalnym. Większość klientów decyduje się poczekać w okolicach wartości średniej, a skrajne przypadki (bardzo szybka lub bardzo długa rezygnacja) występują rzadziej. Dodatkowo stosowane jest kłamrowanie wyników, aby zapobiec generowaniu absurdalnie krótkich lub długich czasów oczekiwania.

3. Rozkład równomierny (`std::uniform_int_distribution`) – wygląd klienta

Każdy wariant wyglądu klienta losowany jest z równym prawdopodobieństwem. W tym projekcie przyjęto dwa warianty, z których każdy ma 50% szansy pojawienia się w kolejce. Rozkład jednolity jest naturalnym wyborem, gdy nie chcemy faworyzować

żadnego modelu wizualnego i chcemy uzyskać równomierne rozmieszczenie wygląków postaci w symulacji.

4. Rozkład dyskretny (std::discrete_distribution) – losowanie typu dania

Do wyboru rodzaju dania stosowany jest rozkład dyskretny z wagami odzwierciedlającymi popularność poszczególnych potraw. Pozwala to symulować realistyczną częstość zamawiania popularnych dań, zamiast przydzielać je z równym prawdopodobieństwem.

5. Rozkład geometryczny (std::geometric_distribution) – wybór klienta rezygnującego z kolejki

Do wyboru, który klient w kolejce zrezygnuje, użyto rozkładu geometrycznego. Faworyzuje on końcówkę kolejki, co realistycznie odzwierciedla zachowanie – osoby stojące najdalej od obsługi częściej decydują się odejść.

6. Rozkład Bernoulliego (std::bernoulli_distribution) – natychmiastowe zajęcie stolika przez klienta

Decyzja o natychmiastowym zajęciu wolnego stolika jest zdarzeniem zero-jedynkowym, więc rozkład Bernoulliego jest naturalnym wyborem. „Prawda” oznacza natychmiastowe zajęcie stolika przez klienta, a „fałsz” – że klient poczeka.

7. Rozkład logarytmicznie normalny (std::lognormal_distribution) – czas jedzenia

Czas jedzenia modelowany jest rozkładem log-normalnym, generującym wartości dodatnie z prawym ogonem – niektórzy klienci mogą jeść znacznie dłużej niż średnia. Klamrowanie zapewnia, że czas pozostaje w rozsądnym zakresie, unikając skrajnych wartości.

8. Rozkład Poissona (std::poisson_distribution) – czas obsługi stolika przez kelnera

Rozkład Poissona stosowany jest do modelowania liczby drobnych czynności kelnera przy obsłudze stolika. Wynik jest ograniczony do sensownego przedziału, co zapewnia realistyczne czasy obsługi.

9. Rozkład gamma (std::gamma_distribution) – czas gotowania

Czas przygotowania potrawy modelowany jest rozkładem gamma, ponieważ jest sumą kilku etapów przygotowania. Rozkład gamma generuje wartości dodatnie z prawym ogonem, dobrze oddając naturalną zmienność w czasie gotowania. Klamrowanie ogranicza wyniki do realistycznych wartości.

3.4. Ekrany aplikacji



Rysunek 3.2. Zdjęcie przedstawiające Menu główne aplikacji



Rysunek 3.3. Zdjęcie przedstawiające uruchomioną symulację



Rysunek 3.4. Zdjęcie przedstawiające ekran ze statystykami, wyliczonymi podczas trwania symulacji

3.5. Opis plików konfiguracyjnych

3.5.1. Plik CMakeLists.txt

Plik CMakeLists.txt jest głównym plikiem konfiguracyjnym używanym przez CMake do budowania projektu. Określa standard C++, zależności od bibliotek zewnętrznych oraz sposób kompilacji i kopiowania plików zasobów po zbudowaniu aplikacji.

Tabela 3.1. Plik CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(RestaurantSimulator LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(SFML_DIR "${CMAKE_SOURCE_DIR}/external/SFML-2.6.2/lib/cmake/SFML")

find_package(SFML 2.6 COMPONENTS system window graphics audio REQUIRED)

file(GLOB_RECURSE SOURCES
    src/*.cpp
)

set(RESOURCE_FILES resources.rc)

add_executable(RestaurantSimulator ${SOURCES} ${RESOURCE_FILES})
```

```

target_include_directories(RestaurantSimulator PRIVATE include)

target_link_libraries(RestaurantSimulator
    sfml-system
    sfml-window
    sfml-graphics
    sfml-audio
)

set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} -mwindows")

add_custom_command(TARGET RestaurantSimulator POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory
    "${CMAKE_SOURCE_DIR}/external/SFML-2.6.2/bin"
    $<TARGET_FILE_DIR:RestaurantSimulator>
    COMMAND ${CMAKE_COMMAND} -E copy_directory
    "${CMAKE_SOURCE_DIR}/dlls"
    $<TARGET_FILE_DIR:RestaurantSimulator>
    COMMAND ${CMAKE_COMMAND} -E copy_directory
    "${CMAKE_SOURCE_DIR}/assets"
    $<TARGET_FILE_DIR:RestaurantSimulator>/assets)

```

W szczególności, plik definiuje minimalną wymaganą wersję CMake (3.16) oraz standard C++17, którego wymaga projekt. Wskazuje katalog z biblioteką SFML 2.6.2 i wyszukuje wszystkie pliki źródłowe w katalogu src/. Tworzony jest plik wykonywalny RestaurantSimulator z uwzględnieniem zasobów (resources.rc), a katalog include/ dodany jest do ścieżek nagłówków. Plik linkuje projekt z bibliotekami SFML (system, window, graphics, audio) i ustawia opcję -mwindows dla linkera.

Dodatkowo, po kompilacji wykonywane są komendy kopiujące:

- pliki DLL SFML z katalogu external/SFML-2.6.2/bin do katalogu z plikiem wykonywalnym,
- pliki DLL z katalogu dlls,
- wszystkie zasoby z katalogu assets do folderu assets obok pliku wykonywalnego.

Dzięki temu projekt po zbudowaniu jest gotowy do uruchomienia bez konieczności ręcznego kopiowania bibliotek czy grafik.

3.5.2. Plik Installer.nsi

Plik RestaurantSimulatorInstaller.nsi to skrypt instalatora używanego przez NSIS (Nullsoft Scriptable Install System). Określa sposób instalacji aplikacji, kopiowanie plików, tworzenie skrótów oraz obsługę odinstalowywania.

Tabela 3.2. Plik Installer.nsi

Name	"Restaurant Simulator"
------	------------------------


```

OutFile "RestaurantSimulatorInstaller.exe"
InstallDir "$PROGRAMFILES\RestaurantSimulator"
Icon "assets\icons\icon.ico"
UninstallIcon "assets\icons\icon.ico"
RequestExecutionLevel admin

Page directory
Page instfiles

Section "Install"

    SetOutPath "$INSTDIR"
    File "build\RestaurantSimulator.exe"
    File /r "build\*.dll"
    SetOutPath "$INSTDIR\assets"
    File /r "assets\*.*)"
    SetOutPath "$INSTDIR"
    File "readme.txt"
    WriteUninstaller "$INSTDIR\Uninstall.exe"
    CreateDirectory "$SMPROGRAMS\RestaurantSimulator"
    CreateShortCut "$SMPROGRAMS\RestaurantSimulator\RestaurantSimulator.lnk"
"$INSTDIR\RestaurantSimulator.exe" "" "$INSTDIR\assets\icons\icon.ico" 0
SW_SHOWNORMAL "" "$INSTDIR"
    CreateShortCut "$DESKTOP\RestaurantSimulator.lnk"
"$INSTDIR\RestaurantSimulator.exe" "" "$INSTDIR\assets\icons\icon.ico" 0
SW_SHOWNORMAL "" "$INSTDIR"
    CreateShortCut "$SMPROGRAMS\RestaurantSimulator\Uninstall.lnk"
"$INSTDIR\Uninstall.exe" "" "$INSTDIR\assets\icons\icon.ico"
    WriteRegStr HKLM
"Software\Microsoft\Windows\CurrentVersion\Uninstall\RestaurantSimulator"
"DisplayName" "RestaurantSimulator"
    WriteRegStr HKLM
"Software\Microsoft\Windows\CurrentVersion\Uninstall\RestaurantSimulator"
"UninstallString" "$INSTDIR\Uninstall.exe"
    WriteRegStr HKLM
"Software\Microsoft\Windows\CurrentVersion\Uninstall\RestaurantSimulator"
"DisplayIcon" "$INSTDIR\assets\icons\icon.ico"

SectionEnd

Section "Uninstall"

    Delete "$INSTDIR\RestaurantSimulator.exe"
    Delete "$INSTDIR\*.dll"
    RMDir /r "$INSTDIR\assets"
    Delete "$INSTDIR\restaurantSimulator.ini"
    Delete "$INSTDIR\stats.txt"
    Delete "$INSTDIR\readme.txt"
    Delete "$INSTDIR\Uninstall.exe"
    Delete "$SMPROGRAMS\RestaurantSimulator\RestaurantSimulator.lnk"
    Delete "$SMPROGRAMS\RestaurantSimulator\Uninstall.lnk"
    RMDir "$SMPROGRAMS\RestaurantSimulator"
    Delete "$DESKTOP\RestaurantSimulator.lnk"
    RMDir "$INSTDIR"
    DeleteRegKey HKLM
"Software\Microsoft\Windows\CurrentVersion\Uninstall\RestaurantSimulator"

SectionEnd

```

W szczegółach, skrypt ustawia nazwę programu, lokalizację docelową instalacji (\$PROGRAMFILES\RestaurantSimulator) oraz ikony dla programu i odinstalowywania. Definiuje strony instalatora: wybór katalogu instalacji oraz postęp instalacji.

W sekcji instalacyjnej (Section "Install") kopiowany jest plik wykonywalny RestaurantSimulator.exe, wszystkie wymagane DLL-e z katalogu build/, a także wszystkie zasoby graficzne i pliki tekstowe do katalogu assets. Tworzony jest odinstalator (Uninstall.exe) oraz skróty w menu Start i na pulpicie z ikoną aplikacji. Skrypt dodaje również wpisy w rejestrze systemowym Windows, umożliwiające późniejsze odinstalowanie programu.

W sekcji odinstalowywania (Section "Uninstall") usuwane są wszystkie pliki programu, DLL-e, zasoby, pliki konfiguracyjne (restaurantSimulator.ini, stats.txt), skróty w menu Start i na pulpicie oraz klucze rejestru. Skrypt zapewnia, że po odinstalowaniu nie pozostaną żadne pliki ani wpisy systemowe związane z aplikacją.

3.5.3. Plik resources.rc

Plik resources.rc jest używany w systemie Windows do definiowania zasobów aplikacji, takich jak ikony, kursory czy menu. W przypadku Restaurant Simulator służy on głównie do przypisania ikony aplikacji.

Tabela 3.3. Plik resources.rc

IDI_ICON1 ICON "assets/icons/icon.ico"
--

Plik zawiera definicję IDI_ICON1 ICON "assets/icons/icon.ico", która przypisuje plik ikony (icon.ico) znajdujący się w katalogu assets/icons do identyfikatora IDI_ICON1. Ta ikona jest następnie wykorzystywana w pliku wykonywalnym programu, instalatorze NSIS oraz skrótach do aplikacji, dzięki czemu aplikacja posiada spójny wygląd ikony zarówno w systemie Windows, jak i w menu Start oraz na pulpicie.

3.5.4. Plik .gitignore

Plik .gitignore służy do wskazania plików i katalogów, które mają być ignorowane przez system kontroli wersji Git. Dzięki temu niepotrzebne pliki tymczasowe, binaria czy zależności zewnętrzne nie trafiają do repozytorium.

Tabela 3.4. Plik .gitignore

.vscode/ build/ external/

dlls/ RestaurantSimulatorInstaller.exe

W szczegółach, plik zawiera następujące wpisy:

- vscode/ – katalog ustawień edytora Visual Studio Code, zawierający konfigurację środowiska programistycznego.
- build/ – katalog wynikowy CMake, w którym znajdują się pliki obiektowe i pliki wykonywalne tworzone podczas kompilacji.
- external/ – katalog zewnętrznych zależności, w tym biblioteka SFML.
- dlls/ – katalog z dynamicznymi bibliotekami potrzebnymi do uruchomienia aplikacji.
- RestaurantSimulatorInstaller.exe – plik wykonywalny instalatora NSIS, który nie powinien być przechowywany w repozytorium, ponieważ jest generowany podczas budowy projektu.

Dzięki tym wpisom repozytorium pozostaje czyste, zawiera jedynie kod źródłowy i pliki potrzebne do jego kompilacji, a nie tymczasowe pliki środowiska czy artefakty kompilacji.

3.6. Opis najważniejszych fragmentów kodu

3.6.1. Plik Game.cpp

Plik Game.cpp implementuje główną klasę Game, która odpowiada za inicjalizację aplikacji, główną pętlę symulacji, przełączanie scen oraz zarządzanie oknem gry. Jest to centralny fragment kodu integrujący wszystkie moduły, takie jak sceny (MainMenu, Level, Summary), zarządzanie postaciami, stołami i daniami.

Tabela 3.5. Plik Game.cpp

```
#include "Game.hpp"
#include "scenes/Level.hpp"
#include "scenes/MainMenu.hpp"
#include "scenes/Summary.hpp"

#include <algorithm>
#include <random>
#include <SFML/Graphics.hpp>

std::mt19937 globalRNG(std::random_device{}());

Game::Game() {
    this->window = new sf::RenderWindow(sf::VideoMode(this->width * this->scaleFactor,
                                                         this->height * this->scaleFactor),
                                         "Restaurant Simulator",
```

```

sf::Style::Titlebar |
sf::Style::Close);
    loadIcon();
    centerWindowOnDesktop();
    this->currentScene = new MainMenu(&this->scaleFactor, &this-
>chefsNumber,
                                &this->waitersNumber,
                                &this->twoChairsTablesNumber,
                                &this->fourChairsTablesNumber,
                                &this-
>timeToEndSimulationInMinutes);
}

Game::~~Game() {
    delete this->window;
    delete this->currentScene;
}

bool Game::loadIcon() {
    sf::Image icon;
    if (icon.loadFromFile("assets/icons/icon.png")) {
        this->window->setIcon(icon.getSize().x, icon.getSize().y,
icon.getPixelsPtr());
        return true;
    }
    return false;
}

void Game::centerWindowOnDesktop() {
    const sf::VideoMode desktopMode = sf::VideoMode::getDesktopMode();
    const sf::Vector2u windowSize = window->getSize();

    const int positionX = std::max(0, (static_cast<int>(desktopMode.width)
-
                                static_cast<int>(windowSize.x)) /
2);
    const int positionY = std::max(0,
(static_cast<int>(desktopMode.height) -
                                static_cast<int>(windowSize.y)) /
2);

    window->setPosition(sf::Vector2i(positionX, positionY));
}

void Game::run() {
    sf::Clock clock;
    while (window->isOpen()) {
        float deltaTime = clock.restart().asSeconds();

        currentScene->update(deltaTime, window);

        if(currentScene->changeScene(&this->newSceneName)) {
            updateScene();
        }

        currentScene->render(window);
        window->display();
    }
}

void Game::updateScene() {

```

```

        if(currentSceneName == newSceneName) {
            int newWindowWidth = width * scaleFactor;
            int newWindowHeight = height * scaleFactor;

            window->setSize(sf::Vector2u(newWindowWidth, newWindowHeight));
            window->setView(sf::View(sf::FloatRect(0.0f, 0.0f,

static_cast<float>(newWindowWidth),

static_cast<float>(newWindowHeight))));
            centerWindowOnDesktop();
            changeScene();
        }
        else {
            changeScene();
            currentSceneName = newSceneName;
        }
    }

void Game::changeScene() {
    delete this->currentScene;

    switch(newSceneName) {
        case ScenesEnum::MAIN_MENU:
            this->currentScene = new MainMenu(&this->scaleFactor, &this->chefsNumber,
                                                &this->waitersNumber,
                                                &this->twoChairsTablesNumber,
                                                &this->fourChairsTablesNumber,
                                                &this->timeToEndSimulationInMinutes);
            break;
        case ScenesEnum::LEVEL:
            this->currentScene = new Level(this->scaleFactor, this->chefsNumber,
                                            this->waitersNumber,
                                            this->twoChairsTablesNumber,
                                            this->fourChairsTablesNumber,
                                            this->timeToEndSimulationInMinutes,
                                            &this->totalCustomersNumberCounter,
                                            &this->resigningCustomersNumberCounter,
                                            &this->insideCustomersNumberCounter,
                                            &this->leavingCustomersNumberCounter,
                                            &this->orderedDishesNumberCounter,
                                            &this->preparedDishesNumberCounter,
                                            &this->servedDishesNumberCounter,
                                            &this->eatenDishesNumberCounter,
                                            &this->droppedDishesNumberCounter);
            break;
    }
}

```

```

        case ScenesEnum::SUMMARY:
            this->currentScene = new Summary(this->scaleFactor, this->chefsNumber,
                                                    this->waitersNumber,
                                                    this->twoChairsTablesNumber,
                                                    this->fourChairsTablesNumber,
                                                    this->timeToEndSimulationInMinutes,
                                                    this->totalCustomersNumberCounter,
                                                    this->resigningCustomersNumberCounter,
                                                    this->insideCustomersNumberCounter,
                                                    this->leavingCustomersNumberCounter,
                                                    this->orderedDishesNumberCounter,
                                                    this->preparedDishesNumberCounter,
                                                    this->servedDishesNumberCounter,
                                                    this->eatenDishesNumberCounter,
                                                    this->droppedDishesNumberCounter);
            break;
        default:
            break;
    }
}

```

W szczegółach plik definiuje konstruktor i destruktor klasy Game, które tworzą okno SFML, ładują ikonę aplikacji i centrowanie okna na pulpicie, a także inicjalizują pierwszą scenę (MainMenu). Metoda run() uruchamia główną pętlę gry, w której wywoływana jest aktualizacja (update) i renderowanie (render) bieżącej sceny oraz sprawdzane jest żądanie zmiany sceny. Przełączanie scen realizowane jest w metodzie updateScene(), która ustawia nowe wymiary okna, aktualizuje widok i wywołuje changeScene(). W tej ostatniej funkcji tworzona jest odpowiednia instancja sceny (MainMenu, Level lub Summary) w zależności od wartości newSceneName. Ponadto plik zawiera funkcje pomocnicze loadIcon() do wczytania ikony okna i centerWindowOnDesktop() do centrowania okna względem ekranu, co poprawia komfort użytkownika. Cała logika zarządzania scenami i główną pętlą gry jest skupiona w tym pliku, dzięki czemu pełni on rolę rdzenia aplikacji, integrującego wszystkie pozostałe moduły.

3.6.2. Plik MainMenu.cpp

Plik MainMenu.cpp implementuje klasę MainMenu, która odpowiada za ekran startowy gry. Zarządza wyświetlaniem tła, tytułu, opcji konfiguracyjnych symulacji, przycisku start

oraz obsługą interakcji użytkownika, w tym kliknięć myszy i efektów wizualnych, takich jak animacje tekstu i blur tła.

Tabela 3.6. Plik MainMenu.cpp

```
#include "scenes/MainMenu.hpp"
#include "utils/InitFileReader.hpp"
#include "utils/InitFileWriter.hpp"

#include <algorithm>
#include <cmath>
#include <string>

MainMenu::MainMenu(int* scaleFactor, int* chefsNumberPtr,
                  int* waitersNumberPtr, int* twoChairsTablesNumberPtr,
                  int* fourChairsTablesNumberPtr, int* simulationTimePtr
                  ) : Scene(*scaleFactor),
                      scaleFactorPtr(scaleFactor),
                      chefsNumberPtr(chefsNumberPtr),
                      waitersNumberPtr(waitersNumberPtr),
                      twoChairsTablesNumberPtr(twoChairsTablesNumberPtr),

fourChairsTablesNumberPtr(fourChairsTablesNumberPtr),
                      simulationTimePtr(simulationTimePtr) {
    loadInitValues();
    texturesLoaded = loadTextures(this->scaleFactor);
    if(fontLoaded) {
        configureTextsStyles();
        updateValuesTexts();
    }
}

MainMenu::~~MainMenu() {
    InitFileWriter initFileWriter(*scaleFactorPtr, *chefsNumberPtr,
                                  *waitersNumberPtr,
                                  *twoChairsTablesNumberPtr,
                                  *fourChairsTablesNumberPtr,
                                  *simulationTimePtr);
    initFileWriter.createConfigFile();
}

void MainMenu::loadInitValues() {
    InitFileReader initFileReader;
    std::unordered_map<std::string, int> configValues =
initFileReader.getConfigValues();
    newScaleFactor = configValues["scaleFactor"];
    newChefsNumber = configValues["chefsNumber"];
    newWaitersNumber = configValues["waitersNumber"];
    newTwoChairsTablesNumber = configValues["twoChairsTablesNumber"];
    newFourChairsTablesNumber = configValues["fourChairsTablesNumber"];
    newSimulationTimeInMinutes =
configValues["timeToEndSimulationInMinutes"];
    if(newScaleFactor != scaleFactor) {
        isSceneToReload = true;
    }
}

bool MainMenu::loadTextures(int scaleFactor) {
    if(!backgroundTexture.loadFromFile("assets/backgrounds/background.png")) {
```

```

        return false;
    }
    backgroundTexture.setSmooth(true);
    backgroundSprite.setTexture(backgroundTexture);
    backgroundSprite.setScale(static_cast<float>(scaleFactor),
static_cast<float>(scaleFactor));

    if(!font.loadFromFile("assets/fonts/determination.ttf")) {
        return false;
    }
    fontLoaded = true;

    return true;
}

void MainMenu::configureTextsStyles() {
    const unsigned int baseOptionTextSize = static_cast<unsigned
int>(fontSize + 2);
    const unsigned int baseTitleTextSize = static_cast<unsigned
int>(fontSize + 12);
    const unsigned int baseStartTextSize = static_cast<unsigned
int>(fontSize + 6);

    const sf::Color labelColor(245, 233, 190);
    const sf::Color valueColor(255, 222, 120);
    const sf::Color outlineColor(58, 58, 80);

    titleText.setFont(font);
    titleText.setCharacterSize(baseTitleTextSize * scaleFactor);
    titleText.setString("Restaurant Simulator");
    titleText.setFillColor(sf::Color(252, 247, 220));
    titleText.setOutlineThickness(1.5f * static_cast<float>(scaleFactor));
    titleText.setOutlineColor(outlineColor);

    scaleFactorLabelText.setFont(font);
    scaleFactorLabelText.setCharacterSize(baseOptionTextSize *
scaleFactor);
    scaleFactorLabelText.setString("Scale Factor:");
    scaleFactorLabelText.setFillColor(labelColor);

    scaleFactorLabelText.setOutlineThickness(static_cast<float>(scaleFactor));
    scaleFactorLabelText.setOutlineColor(outlineColor);

    scaleFactorValueText.setFont(font);
    scaleFactorValueText.setCharacterSize(baseOptionTextSize *
scaleFactor);
    scaleFactorValueText.setFillColor(valueColor);

    scaleFactorValueText.setOutlineThickness(static_cast<float>(scaleFactor));
    scaleFactorValueText.setOutlineColor(outlineColor);

    chefsNumberLabelText.setFont(font);
    chefsNumberLabelText.setCharacterSize(baseOptionTextSize *
scaleFactor);
    chefsNumberLabelText.setString("Chefs Number:");
    chefsNumberLabelText.setFillColor(labelColor);

    chefsNumberLabelText.setOutlineThickness(static_cast<float>(scaleFactor));
    chefsNumberLabelText.setOutlineColor(outlineColor);

    chefsNumberValueText.setFont(font);

```



```

        chefsNumberValueText.setCharacterSize(baseOptionTextSize *
scaleFactor);
        chefsNumberValueText.setFillColor(valueColor);

chefsNumberValueText.setOutlineThickness(static_cast<float>(scaleFactor));
        chefsNumberValueText.setOutlineColor(outlineColor);

        waitersNumberLabelText.setFont(font);
        waitersNumberLabelText.setCharacterSize(baseOptionTextSize *
scaleFactor);
        waitersNumberLabelText.setString("Waiters Number:");
        waitersNumberLabelText.setFillColor(labelColor);

waitersNumberLabelText.setOutlineThickness(static_cast<float>(scaleFactor))
;
        waitersNumberLabelText.setOutlineColor(outlineColor);

        waitersNumberValueText.setFont(font);
        waitersNumberValueText.setCharacterSize(baseOptionTextSize *
scaleFactor);
        waitersNumberValueText.setFillColor(valueColor);

waitersNumberValueText.setOutlineThickness(static_cast<float>(scaleFactor))
;
        waitersNumberValueText.setOutlineColor(outlineColor);

        twoChairsTablesNumberLabelText.setFont(font);
        twoChairsTablesNumberLabelText.setCharacterSize(baseOptionTextSize *
scaleFactor);
        twoChairsTablesNumberLabelText.setString("2 Chairs Tables Number:");
        twoChairsTablesNumberLabelText.setFillColor(labelColor);

twoChairsTablesNumberLabelText.setOutlineThickness(static_cast<float>(scale
Factor));
        twoChairsTablesNumberLabelText.setOutlineColor(outlineColor);

        twoChairsTablesNumberValueText.setFont(font);
        twoChairsTablesNumberValueText.setCharacterSize(baseOptionTextSize *
scaleFactor);
        twoChairsTablesNumberValueText.setFillColor(valueColor);

twoChairsTablesNumberValueText.setOutlineThickness(static_cast<float>(scale
Factor));
        twoChairsTablesNumberValueText.setOutlineColor(outlineColor);

        fourChairsTablesNumberLabelText.setFont(font);
        fourChairsTablesNumberLabelText.setCharacterSize(baseOptionTextSize *
scaleFactor);
        fourChairsTablesNumberLabelText.setString("4 Chairs Tables Number:");
        fourChairsTablesNumberLabelText.setFillColor(labelColor);

fourChairsTablesNumberLabelText.setOutlineThickness(static_cast<float>(scal
eFactor));
        fourChairsTablesNumberLabelText.setOutlineColor(outlineColor);

        fourChairsTablesNumberValueText.setFont(font);
        fourChairsTablesNumberValueText.setCharacterSize(baseOptionTextSize *
scaleFactor);
        fourChairsTablesNumberValueText.setFillColor(valueColor);

fourChairsTablesNumberValueText.setOutlineThickness(static_cast<float>(scal

```

```

eFactor));
    fourChairsTablesNumberValueText.setOutlineColor(outlineColor);

    simulationTimeLabelText.setFont(font);
    simulationTimeLabelText.setCharacterSize(baseOptionTextSize *
scaleFactor);
    simulationTimeLabelText.setString("Simulation Time:");
    simulationTimeLabelText.setFillColor(labelColor);

simulationTimeLabelText.setOutlineThickness(static_cast<float>(scaleFactor)
);
    simulationTimeLabelText.setOutlineColor(outlineColor);

    simulationTimeValueText.setFont(font);
    simulationTimeValueText.setCharacterSize(baseOptionTextSize *
scaleFactor);
    simulationTimeValueText.setFillColor(valueColor);

simulationTimeValueText.setOutlineThickness(static_cast<float>(scaleFactor)
);
    simulationTimeValueText.setOutlineColor(outlineColor);

    startText.setFont(font);
    startText.setCharacterSize(baseStartTextSize * scaleFactor);
    startText.setString("START");
    startText.setFillColor(sf::Color(255, 236, 141));
    startText.setOutlineThickness(1.3f * static_cast<float>(scaleFactor));
    startText.setOutlineColor(outlineColor);
}

void MainMenu::changeAnimation(float deltaTime) {
    animTime += deltaTime;
    animTime = std::fmod(animTime + deltaTime, 1000.0f);

    const float pulse = 1.0f + 0.035f * std::sin(animTime * 2.7f);
    titleText.setScale(pulse, pulse);

    const float wave = 0.5f + 0.5f * std::sin(animTime * 4.0f);
    const sf::Uint8 green = static_cast<sf::Uint8>(200.0f + 40.0f * wave);
    titleText.setFillColor(sf::Color(252, green, 180));

    const float startPulse = 0.95f + 0.08f * (0.5f + 0.5f *
std::sin(animTime * 5.0f));
    startText.setScale(startPulse, startPulse);
}

void MainMenu::updateValuesTexts() {
    *scaleFactorPtr = std::max(1, std::min(newScaleFactor, 9));
    *chefsNumberPtr = std::max(1, std::min(newChefsNumber, 3));
    *waitersNumberPtr = std::max(1, std::min(newWaitersNumber, 3));
    *twoChairsTablesNumberPtr = std::max(0,
std::min(newTwoChairsTablesNumber, 8));
    *fourChairsTablesNumberPtr = std::max(0,
std::min(newFourChairsTablesNumber, 3));
    *simulationTimePtr = std::max(5, std::min(newSimulationTimeInMinutes,
60));

    scaleFactorValueText.setString(std::to_string(*scaleFactorPtr));
    chefsNumberValueText.setString(std::to_string(*chefsNumberPtr));
    waitersNumberValueText.setString(std::to_string(*waitersNumberPtr));
}

```

```

twoChairsTablesNumberValueText.setString(std::to_string(*twoChairsTablesNum
berPtr));

fourChairsTablesNumberValueText.setString(std::to_string(*fourChairsTablesN
umberPtr));
    simulationTimeValueText.setString(std::to_string(*simulationTimePtr) +
" min");
}

void MainMenu::updateTextsPositions() {

    const auto scaled = [this](float referenceValue) {
        return referenceValue * static_cast<float>(scaleFactor);
    };

    const float centerX = scaled(151.0f);
    const float titleY = scaled(34.0f);
    const float valueSpacing = scaled(12.0f);
    const float startY = scaled(200.0f);

    const sf::FloatRect titleBounds = titleText.getLocalBounds();
    titleText.setOrigin(titleBounds.left + titleBounds.width / 2.0f,
        titleBounds.top + titleBounds.height / 2.0f);
    titleText.setPosition(centerX, titleY);

    const sf::FloatRect startBounds = startText.getLocalBounds();
    startText.setOrigin(startBounds.left + startBounds.width / 2.0f,
        startBounds.top + startBounds.height / 2.0f);
    startText.setPosition(centerX, startY);

    const float titleBottom = titleY + titleBounds.height / 2.0f;
    const float startTop = startY - startBounds.height / 2.0f;

    auto optionRowHeight = [](const sf::Text& labelText, const sf::Text&
valueText) {
        const sf::FloatRect labelBounds = labelText.getLocalBounds();
        const sf::FloatRect valueBounds = valueText.getLocalBounds();
        return std::max(labelBounds.height, valueBounds.height);
    };

    const float scaleFactorRowHeight =
optionRowHeight(scaleFactorLabelText, scaleFactorValueText);
    const float chefsNumberRowHeight =
optionRowHeight(chefsNumberLabelText, chefsNumberValueText);
    const float waitersNumberRowHeight =
optionRowHeight(waitersNumberLabelText, waitersNumberValueText);
    const float twoChairsTablesRowHeight =
optionRowHeight(twoChairsTablesNumberLabelText,
twoChairsTablesNumberValueText);
    const float fourChairsTablesRowHeight =
optionRowHeight(fourChairsTablesNumberLabelText,
fourChairsTablesNumberValueText);
    const float simulationTimeRowHeight =
optionRowHeight(simulationTimeLabelText,
simulationTimeValueText);

    const float optionsTotalHeight = scaleFactorRowHeight +
chefsNumberRowHeight +

```

```

        waitersNumberRowHeight +
twoChairsTablesRowHeight +
        fourChairsTablesRowHeight +
simulationTimeRowHeight;
    const float equalGap = (startTop - titleBottom - optionsTotalHeight) /
7.0f;

    const float scaleFactorOptionTop = titleBottom + equalGap;
    const float chefsNumberOptionTop = scaleFactorOptionTop +
scaleFactorRowHeight + equalGap;
    const float waitersNumberOptionTop = chefsNumberOptionTop +
chefsNumberRowHeight + equalGap;
    const float twoChairsTablesOptionTop = waitersNumberOptionTop +
waitersNumberRowHeight + equalGap;
    const float fourChairsTablesOptionTop = twoChairsTablesOptionTop +
twoChairsTablesRowHeight + equalGap;
    const float simulationTimeOptionTop = fourChairsTablesOptionTop +
fourChairsTablesRowHeight + equalGap;

    auto positionOption = [centerX, valueSpacing](sf::Text& labelText,
sf::Text& valueText,
        float topY) {
        const sf::FloatRect labelBounds = labelText.getLocalBounds();
        const sf::FloatRect valueBounds = valueText.getLocalBounds();

        const float totalWidth = labelBounds.width + valueSpacing +
valueBounds.width;
        const float leftX = centerX - (totalWidth / 2.0f);

        labelText.setPosition(leftX - labelBounds.left, topY -
labelBounds.top);
        valueText.setPosition(leftX + labelBounds.width + valueSpacing -
valueBounds.left,
            topY - valueBounds.top);
    };

    positionOption(scaleFactorLabelText, scaleFactorValueText,
scaleFactorOptionTop);
    positionOption(chefsNumberLabelText, chefsNumberValueText,
chefsNumberOptionTop);
    positionOption(waitersNumberLabelText, waitersNumberValueText,
waitersNumberOptionTop);
    positionOption(twoChairsTablesNumberLabelText,
twoChairsTablesNumberValueText,
twoChairsTablesOptionTop);
    positionOption(fourChairsTablesNumberLabelText,
fourChairsTablesNumberValueText,
fourChairsTablesOptionTop);
    positionOption(simulationTimeLabelText, simulationTimeValueText,
simulationTimeOptionTop);
}

bool MainMenu::isMouseOverText(sf::RenderWindow* window, sf::Text text) {
    const sf::Vector2i mousePositions = sf::Mouse::getPosition(*window);
    const sf::Vector2f mouseWindowPositions = window->
mapPixelToCoords(mousePositions);
    return text.getGlobalBounds().contains(mouseWindowPositions);
}

void MainMenu::updateHoverState(sf::RenderWindow* window) {
    const sf::Color normalValueColor(255, 222, 120);

```

```

const sf::Color hoverValueColor(255, 255, 190);
const sf::Color normalStartColor(255, 236, 141);
const sf::Color hoverStartColor(255, 255, 210);

scaleFactorValueText.setFillColor(isMouseOverText(window,
scaleFactorValueText) ?
                                hoverValueColor : normalValueColor);
chefsNumberValueText.setFillColor(isMouseOverText(window,
chefsNumberValueText) ?
                                hoverValueColor : normalValueColor);
waitersNumberValueText.setFillColor(isMouseOverText(window,
waitersNumberValueText) ?
                                hoverValueColor :
normalValueColor);
fourChairsTablesNumberValueText.setFillColor(isMouseOverText(window,
fourChairsTablesNumberValueText) ?
                                hoverValueColor :
normalValueColor);
twoChairsTablesNumberValueText.setFillColor(isMouseOverText(window,
twoChairsTablesNumberValueText) ?
                                hoverValueColor :
normalValueColor);
simulationTimeValueText.setFillColor(isMouseOverText(window,
simulationTimeValueText) ?
                                hoverValueColor :
normalValueColor);
startText.setFillColor(isMouseOverText(window, startText) ?
hoverStartColor :
                                normalStartColor);
}

void MainMenu::handleMouseClicked(sf::RenderWindow* window, bool isLeftClick)
{
    int value = isLeftClick ? 1 : -1;
    if(isMouseOverText(window, scaleFactorValueText)) {
        newScaleFactor = std::max(1, std::min(newScaleFactor + value, 9));
        updateValuesTexts();
        isSceneToReload = true;
        return;
    }
    if(isMouseOverText(window, chefsNumberValueText)) {
        newChefsNumber = std::max(1, std::min(newChefsNumber + value, 3));
        updateValuesTexts();
        return;
    }
    if(isMouseOverText(window, waitersNumberValueText)) {
        newWaitersNumber = std::max(1, std::min(newWaitersNumber + value,
3));
        updateValuesTexts();
        return;
    }
    if(isMouseOverText(window, twoChairsTablesNumberValueText)) {
        newTwoChairsTablesNumber = std::max(0,
std::min(newTwoChairsTablesNumber + value, 8));
        updateValuesTexts();
        return;
    }
    if(isMouseOverText(window, fourChairsTablesNumberValueText)) {
        newFourChairsTablesNumber = std::max(0,
std::min(newFourChairsTablesNumber + value, 3));
        updateValuesTexts();
    }
}

```

```

        return;
    }
    if(isMouseOverText(window, simulationTimeValueText)) {
        newSimulationTimeInMinutes = std::max(5,
std::min(newSimulationTimeInMinutes +
                                                (value * 5),
60));
        updateValuesTexts();
        return;
    }
    if(isMouseOverText(window, startText)) {
        isSceneToChange = true;
    }
}

void MainMenu::drawBackgroundWithBlur(sf::RenderWindow* window) {
    const sf::Vector2u windowSize = window->getSize();
    const float downsampleFactor = 1.45f;
    const unsigned int reducedWidth = std::max(1u, static_cast<unsigned
int>(
        std::round(static_cast<float>(windowSize.x) / downsampleFactor)));
    const unsigned int reducedHeight = std::max(1u, static_cast<unsigned
int>(
        std::round(static_cast<float>(windowSize.y) / downsampleFactor)));

    if(blurTextureSize.x != reducedWidth || blurTextureSize.y !=
reducedHeight) {
        if (!blurRenderTexture.create(reducedWidth, reducedHeight)) {
            backgroundSprite.setPosition(0.0f, 0.0f);
            backgroundSprite.setColor(sf::Color::White);
            window->draw(backgroundSprite);
            return;
        }
        blurRenderTexture.setSmooth(true);
        blurTextureSize = sf::Vector2u(reducedWidth, reducedHeight);
    }

    sf::Sprite reducedBackground = backgroundSprite;
    reducedBackground.setPosition(0.0f, 0.0f);
    reducedBackground.setScale(
        backgroundSprite.getScale().x * (static_cast<float>(reducedWidth) /
static_cast<float>(windowSize.x)),
        backgroundSprite.getScale().y * (static_cast<float>(reducedHeight)
/
static_cast<float>(windowSize.y)));

    blurRenderTexture.clear(sf::Color::Transparent);
    blurRenderTexture.draw(reducedBackground);
    blurRenderTexture.display();

    backgroundSprite.setPosition(0.0f, 0.0f);
    backgroundSprite.setColor(sf::Color::White);
    window->draw(backgroundSprite);

    blurSprite.setTexture(blurRenderTexture.getTexture(), true);
    blurSprite.setPosition(0.0f, 0.0f);
    blurSprite.setScale(static_cast<float>(windowSize.x) /
static_cast<float>(reducedWidth),
static_cast<float>(windowSize.y) /
static_cast<float>(reducedHeight));

```

```

        blurSprite.setColor(sf::Color(255, 255, 255, 80));
        window->draw(blurSprite);

        sf::RectangleShape
blurVeil(sf::Vector2f(static_cast<float>(windowSize.x),
static_cast<float>(windowSize.y)));
        blurVeil.setFillColor(sf::Color(18, 18, 24, 10));
        window->draw(blurVeil);
    }

void MainMenu::update(float deltaTime, sf::RenderWindow* window) {
    sf::Event event;
    while(window->pollEvent(event)) {
        if(event.type == sf::Event::Closed) {
            window->close();
        }
        if(event.type == sf::Event::MouseButtonPressed &&
            event.mouseButton.button == sf::Mouse::Left) {
            handleMouseClicked(window, true);
        }
        if(event.type == sf::Event::MouseButtonPressed &&
            event.mouseButton.button == sf::Mouse::Right) {
            handleMouseClicked(window, false);
        }
    }
    changeAnimation(deltaTime);
    updateTextsPositions();
    updateHoverState(window);
}

bool MainMenu::changeScene(enum ScenesEnum* sceneName) {
    if(isSceneToReload) {
        *sceneName = ScenesEnum::MAIN_MENU;
        return true;
    }
    if(isSceneToChange) {
        *sceneName = ScenesEnum::LEVEL;
        return true;
    }
    return false;
}

void MainMenu::render(sf::RenderWindow* window) {
    if(texturesLoaded) {
        drawBackgroundWithBlur(window);
    }

    if(fontLoaded) {
        window->draw(titleText);
        window->draw(scaleFactorLabelText);
        window->draw(scaleFactorValueText);
        window->draw(chefsNumberLabelText);
        window->draw(chefsNumberValueText);
        window->draw(waitersNumberLabelText);
        window->draw(waitersNumberValueText);
        window->draw(twoChairsTablesNumberLabelText);
        window->draw(twoChairsTablesNumberValueText);
        window->draw(fourChairsTablesNumberLabelText);
        window->draw(fourChairsTablesNumberValueText);
        window->draw(simulationTimeLabelText);
    }
}

```

```

        window->draw(simulationTimeValueText);
        window->draw(startText);
    }
}

```

W szczegółach plik definiuje konstruktor i destruktor klasy, które odpowiednio ładują wartości początkowe z pliku konfiguracyjnego (InitFileReader), wczytują tekstury i fonty oraz zapisują zmienione ustawienia przy zamykaniu menu (InitFileWriter). Funkcje loadTextures() i configureTextsStyles() przygotowują wszystkie elementy graficzne interfejsu, w tym etykiety i wartości opcji symulacji (np. liczba kucharzy, kelnerów, stolików, czas trwania symulacji).

Plik zawiera również mechanizmy interakcji użytkownika: update() obsługuje zdarzenia SFML, w tym kliknięcia lewym i prawym przyciskiem myszy, handleMouseClicked() zmienia wartości opcji w zależności od kliknięcia, a updateHoverState() zmienia kolory tekstów w momencie najechania kursorem. Animacje tytułu i przycisku start realizowane są w changeAnimation(), a pozycjonowanie tekstów w updateTextsPositions(). Metoda drawBackgroundWithBlur() tworzy efekt rozmycia tła dla estetycznego wyglądu menu.

Na końcu render() odpowiada za rysowanie całego interfejsu w oknie SFML, uwzględniając zarówno tło z efektem blur, jak i wszystkie napisy oraz wartości opcji. Funkcja changeScene() określa, kiedy należy przełączyć scenę na symulację gry (Level) lub przeładować menu w przypadku zmiany skali okna. Plik pełni kluczową rolę w interakcji z użytkownikiem i konfiguracji parametrów symulacji przed rozpoczęciem gry.

3.6.3. Plik Level.cpp

Plik Level.cpp implementuje klasę Level, która zarządza główną sceną symulacji restauracji, w tym postaciami (kucharze, kelnerzy, klienci), stołami, potrawami oraz interfejsem użytkownika (czas i mnożnik prędkości). Odpowiada zarówno za logikę gry, jak i renderowanie wszystkich elementów w odpowiedniej kolejności na ekranie.

Tabela 3.7. Plik Level.cpp

```

#include "scenes/Level.hpp"

#include <SFML/Graphics.hpp>
#include <iomanip>
#include <sstream>
#include <algorithm>
#include <vector>
#include <set>

Level::Level(int scaleFactor, int chefsNumber, int waitersNumber,

```



```

        int twoChairsTablesNumber, int fourChairsTablesNumber,
        int timeToEndSimulationInMinutes,
        int* totalCustomersNumberCounterPtr,
        int* resigningCustomersNumberCounterPtr,
        int* insideCustomersNumberCounterPtr,
        int* leavingCustomersNumberCounterPtr,
        int* orderedDishesNumberCounterPtr,
        int* preparedDishesNumberCounterPtr,
        int* servedDishesNumberCounterPtr,
        int* eatenDishesNumberCounterPtr,
        int* droppedDishesNumberCounterPtr
    ) : Scene(scaleFactor),
        timeToEndSimulationInMinutes(timeToEndSimulationInMinutes),

totalCustomersNumberCounterPtr(totalCustomersNumberCounterPtr),

resigningCustomersNumberCounterPtr(resigningCustomersNumberCounterPtr),

insideCustomersNumberCounterPtr(insideCustomersNumberCounterPtr),

leavingCustomersNumberCounterPtr(leavingCustomersNumberCounterPtr),

orderedDishesNumberCounterPtr(orderedDishesNumberCounterPtr),

preparedDishesNumberCounterPtr(preparedDishesNumberCounterPtr),
        servedDishesNumberCounterPtr(servedDishesNumberCounterPtr),
        eatenDishesNumberCounterPtr(eatenDishesNumberCounterPtr),

droppedDishesNumberCounterPtr(droppedDishesNumberCounterPtr) {
    tileWidth *= scaleFactor;
    tileHeight *= scaleFactor;
    dishesManager = new DishesManager(scaleFactor);
    tablesManager = new TablesManager(scaleFactor, twoChairsTablesNumber,
fourChairsTablesNumber);
    charactersManager = new CharactersManager(scaleFactor, tileWidth,
tileHeight,
                                                chefsNumber, waitersNumber,
dishesManager);
    pathFinder = new Pathfinder(this);
    this->texturesLoaded = loadTextures(scaleFactor);
    if(fontLoaded) {
        configureTextsStyles();
        updateValuesTexts();
    }
}

Level::~Level() {
    charactersManager->getStatistics(totalCustomersNumberCounterPtr,
resigningCustomersNumberCounterPtr,
                                insideCustomersNumberCounterPtr,
leavingCustomersNumberCounterPtr,
                                orderedDishesNumberCounterPtr,
preparedDishesNumberCounterPtr,
                                servedDishesNumberCounterPtr,
eatenDishesNumberCounterPtr,
                                droppedDishesNumberCounterPtr);

    delete dishesManager;
    delete tablesManager;
    delete charactersManager;
    delete pathFinder;
}

```

```

bool Level::loadTextures(int scaleFactor) {
    if(!backgroundTexture.loadFromFile("assets/interiors/background.png"))
    {
        return false;
    }
    backgroundSprite.setTexture(backgroundTexture);
    backgroundSprite.setScale(static_cast<float>(scaleFactor),
static_cast<float>(scaleFactor));

    if(!kitchenWallsTexture.loadFromFile("assets/interiors/kitchen_walls.png"))
    {
        return false;
    }
    kitchenWallsSprite.setTexture(kitchenWallsTexture);
    kitchenWallsSprite.setScale(static_cast<float>(scaleFactor),
static_cast<float>(scaleFactor));

    if(!entranceWallsTexture.loadFromFile("assets/interiors/entrance_walls.png"
)) {
        return false;
    }
    entranceWallsSprite.setTexture(entranceWallsTexture);
    entranceWallsSprite.setScale(static_cast<float>(scaleFactor),
static_cast<float>(scaleFactor));

    if(!flowersTexture.loadFromFile("assets/interiors/flowers.png")) {
        return false;
    }
    flowersSprite.setTexture(flowersTexture);
    flowersSprite.setScale(static_cast<float>(scaleFactor),
static_cast<float>(scaleFactor));

    if(!font.loadFromFile("assets/fonts/determination.ttf")) {
        return false;
    }
    fontLoaded = true;

    return true;
}

void Level::configureTextsStyles() {
    const sf::Color labelColor(245, 233, 190);
    const sf::Color valueColor(255, 222, 120);
    const sf::Color outlineColor(58, 58, 80);

    timeLabelText.setFont(font);
    timeLabelText.setCharacterSize(fontSize * scaleFactor);
    timeLabelText.setString("Remaining Time:");
    timeLabelText.setFillColor(labelColor);
    timeLabelText.setOutlineThickness(0.8f *
static_cast<float>(scaleFactor));
    timeLabelText.setOutlineColor(outlineColor);

    timeValueText.setFont(font);
    timeValueText.setCharacterSize(fontSize * scaleFactor);
    timeValueText.setFillColor(valueColor);
    timeValueText.setOutlineThickness(0.8f *
static_cast<float>(scaleFactor));

```

```

timeValueText.setOutlineColor(outlineColor);

speedMultiplierLabelText.setFont(font);
speedMultiplierLabelText.setCharacterSize(fontSize * scaleFactor);
speedMultiplierLabelText.setString("Speed Multiplier:");
speedMultiplierLabelText.setFillColor(labelColor);
speedMultiplierLabelText.setOutlineThickness(0.8f *
static_cast<float>(scaleFactor));
speedMultiplierLabelText.setOutlineColor(outlineColor);

speedMultiplierValueText.setFont(font);
speedMultiplierValueText.setCharacterSize(fontSize * scaleFactor);
speedMultiplierValueText.setFillColor(valueColor);
speedMultiplierValueText.setOutlineThickness(0.8f *
static_cast<float>(scaleFactor));
speedMultiplierValueText.setOutlineColor(outlineColor);
}

void Level::updateValuesTexts() {
    int minutes = std::max(0, minutesNumber);
    int seconds = std::max(0, secondsNumber % 60);
    std::ostringstream oss;
    oss << std::setw(2) << std::setfill('0') << minutes << ":"
        << std::setw(2) << std::setfill('0') << seconds;
    timeValueText.setString(oss.str());

    speedMultiplierValueText.setString(std::string("x") +
std::to_string(speedMultiplier));
}

void Level::updateTextsPositions() {
    const float margin = 8.0f * static_cast<float>(scaleFactor);

    const sf::FloatRect timeLabelBounds = timeLabelText.getLocalBounds();
    timeLabelText.setPosition(margin - timeLabelBounds.left, margin -
timeLabelBounds.top);

    const sf::FloatRect timeValueBounds = timeValueText.getLocalBounds();
    timeValueText.setPosition(margin + timeLabelBounds.width + 6.0f *
scaleFactor - timeValueBounds.left,
margin - timeValueBounds.top);

    const float rightEdge = static_cast<float>(302 * scaleFactor) - margin;
    const sf::FloatRect labelBounds =
speedMultiplierLabelText.getLocalBounds();
    const sf::FloatRect valueBounds =
speedMultiplierValueText.getLocalBounds();
    const float spacing = 6.0f * static_cast<float>(scaleFactor);
    const float totalWidth = labelBounds.width + spacing +
valueBounds.width;
    const float leftX = rightEdge - totalWidth;

    speedMultiplierLabelText.setPosition(leftX - labelBounds.left, margin -
labelBounds.top);
    speedMultiplierValueText.setPosition(leftX + labelBounds.width +
spacing - valueBounds.left,
margin - valueBounds.top);
}

bool Level::isMouseOverText(sf::RenderWindow* window, sf::Text text) {
    const sf::Vector2i mousePositions = sf::Mouse::getPosition(*window);

```

```

        const sf::Vector2f mouseWindowPositions = window->
>mapPixelToCoords(mousePositions);
        return text.getGlobalBounds().contains(mouseWindowPositions);
    }

void Level::updateHoverState(sf::RenderWindow* window) {
    const sf::Color normalValueColor(255, 222, 120);
    const sf::Color hoverValueColor(255, 255, 190);

    speedMultiplierValueText.setFillColor(isMouseOverText(window,
speedMultiplierValueText) ?
                                hoverValueColor :
normalValueColor);
}

void Level::handleMouseClicked(sf::RenderWindow* window, bool isLeftClick) {
    int value = isLeftClick ? 1 : -1;
    if(isMouseOverText(window, speedMultiplierValueText)) {
        speedMultiplier = std::max(1, std::min(speedMultiplier + value,
60));
        updateValuesTexts();
    }
}

void Level::update(float deltaTime, sf::RenderWindow* window) {
    (void)window;
    sf::Event event;
    while(window->pollEvent(event)) {
        if(event.type == sf::Event::Closed)
            window->close();
        if(event.type == sf::Event::MouseButtonPressed) {
            if(event.mouseButton.button == sf::Mouse::Left) {
                handleClick(window, true);
            }
            else if(event.mouseButton.button == sf::Mouse::Right) {
                handleClick(window, false);
            }
        }
    }

    deltaTime *= speedMultiplier;

    this->charactersManager->update(deltaTime, scaleFactor,
                                tileWidth, tileHeight,
                                dishesManager, tablesManager,
                                pathFinder);

    simulationTime += deltaTime;

    float remainingTimeInMinutes = timeToEndSimulationInMinutes -
(simulationTime / 60.0f);
    remainingTimeInMinutes = std::max(0.0f, remainingTimeInMinutes);

    minutesNumber = static_cast<int>(remainingTimeInMinutes);
    secondsNumber = static_cast<int>((remainingTimeInMinutes -
minutesNumber) * 60);

    if(remainingTimeInMinutes <= 0) {
        isSceneToChange = true;
    }
    updateValuesTexts();
    updateTextsPositions();
}

```

```

        updateHoverState(window);
    }

    bool Level::changeScene(enum ScenesEnum* sceneName) {
        if(isSceneToChange) {
            *sceneName = ScenesEnum::SUMMARY;
            return true;
        }
        return false;
    }

    void Level::render(sf::RenderWindow* window) {
        if(texturesLoaded) {
            window->draw(backgroundSprite);
            window->draw(flowersSprite);
            this->charactersManager->renderChefs(window);
            window->draw(kitchenWallsSprite);
            this->dishesManager->renderReadyDish(window);

            enum class RenderType {
                Table,
                Dish,
                Character
            };

            struct RenderItem {
                float sortY = 0.0f;
                RenderType type = RenderType::Table;
                void* ptr = nullptr;
            };

            std::vector<RenderItem> renderItems;
            const std::vector<Table*> tables = tablesManager->getTables();
            const std::vector<Character*> characters = charactersManager-
>getWaitersAndInsideCustomers();
            const std::vector<Dish*>& dishes = dishesManager-
>getDishesOnTables();

            renderItems.reserve(tables.size() + characters.size() +
dishes.size());
            for(Table* table : tables) {
                renderItems.push_back(RenderItem{table->getSortY() *
static_cast<float>(scaleFactor),
                                                                    RenderType::Table, table});
            }
            std::set<std::pair<float, float>> dishPositions;
            for(Dish* dish : dishes) {
                Positions pos = dish->getPositions();
                if(dishPositions.insert({pos.xPos, pos.yPos}).second) {
                    renderItems.push_back(RenderItem{dish->getYPos(),
RenderType::Dish, dish});
                }
            }
            for(Character* character : characters) {
                renderItems.push_back(RenderItem{character->getSortY(),
RenderType::Character, character});
            }

            std::stable_sort(renderItems.begin(), renderItems.end(),
                            [](const RenderItem& left, const RenderItem&
right) {

```

```

        return left.sortY < right.sortY;
    });

    for(const RenderItem& item : renderItems) {
        if(item.type == RenderType::Table) {
            static_cast<Table*>(item.ptr)->render(window);
        }
        else if(item.type == RenderType::Dish) {
            static_cast<Dish*>(item.ptr)->render(window);
        }
        else {
            static_cast<Character*>(item.ptr)->render(window);
        }
    }
    window->draw(entranceWallsSprite);
    this->charactersManager-
>renderWaitingResigningInsideLeavingCustomers(window);

    }

    if(fontLoaded) {
        window->draw(timeLabelText);
        window->draw(timeValueText);
        window->draw(speedMultiplierLabelText);
        window->draw(speedMultiplierValueText);
    }
}

bool Level::isValidPositions(Positions positions) {
    if(positions.xPos >= (151.0f * scaleFactor) &&
        positions.xPos <= (280.0f * scaleFactor) &&
        positions.yPos == (24.0f * scaleFactor)) {
        return true;
    }
    if(positions.xPos >= (151.0f * scaleFactor)
        && positions.xPos <= (212.0f * scaleFactor)
        && positions.yPos > (24.0f * scaleFactor)
        && positions.yPos <= (56.0f * scaleFactor)) {
        return true;
    }
    if(positions.xPos >= (151.0f * scaleFactor)
        && positions.xPos <= (212.0f * scaleFactor)
        && positions.yPos > (24.0f * scaleFactor)
        && positions.yPos <= (56.0f * scaleFactor)) {
        return true;
    }
    if(positions.xPos >= (70.0f * scaleFactor)
        && positions.xPos <= (88.0f * scaleFactor)
        && positions.yPos == (72.0f * scaleFactor)) {
        return true;
    }
    if(positions.xPos > (151.0f * scaleFactor)
        && positions.xPos <= (280.0f * scaleFactor)
        && positions.yPos == (64.0f * scaleFactor)) {
        return true;
    }
    if(positions.xPos >= (151.0f * scaleFactor)
        && positions.xPos <= (212.0f * scaleFactor)
        && positions.yPos > (56.0f * scaleFactor)
        && positions.yPos <= (72.0f * scaleFactor)) {
        return true;
    }
}

```

```

}
if(positions.xPos >= (7.0f * scaleFactor)
    && positions.xPos <= (212.0f * scaleFactor)
    && positions.yPos == (88.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (70.0f * scaleFactor)
    && positions.xPos <= (88.0f * scaleFactor)
    && positions.yPos > (72.0f * scaleFactor)
    && positions.yPos <= (88.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (151.0f * scaleFactor)
    && positions.xPos <= (212.0f * scaleFactor)
    && positions.yPos > (72.0f * scaleFactor)
    && positions.yPos <= (88.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (52.0f * scaleFactor)
    && positions.xPos <= (280.0f * scaleFactor)
    && positions.yPos == (104.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (52.0f * scaleFactor)
    && positions.xPos <= (212.0f * scaleFactor)
    && positions.yPos > (88.0f * scaleFactor)
    && positions.yPos <= (104.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (7.0f * scaleFactor)
    && positions.xPos <= (212.0f * scaleFactor)
    && positions.yPos == (112.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (52.0f * scaleFactor)
    && positions.xPos <= (212.0f * scaleFactor)
    && positions.yPos > (104.0f * scaleFactor)
    && positions.yPos <= (112.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (101.0f * scaleFactor)
    && positions.xPos <= (164.0f * scaleFactor)
    && positions.yPos > (120.0f * scaleFactor)
    && positions.yPos <= (136.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (7.0f * scaleFactor)
    && positions.xPos <= (280.0f * scaleFactor)
    && positions.yPos == (152.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (101.0f * scaleFactor)
    && positions.xPos <= (164.0f * scaleFactor)
    && positions.yPos > (136.0f * scaleFactor)
    && positions.yPos <= (152.0f * scaleFactor)) {
    return true;
}
if(positions.xPos >= (131.0f * scaleFactor)
    && positions.xPos <= (171.0f * scaleFactor)
    && positions.yPos > (152.0f * scaleFactor)
    && positions.yPos <= (168.0f * scaleFactor)) {

```

```

        return true;
    }
    return false;
}

```

Konstruktor klasy Level inicjalizuje menedżery postaci, stołów i potraw, ładuje tekstury tła i fontu oraz ustawia style tekstów interfejsu. Destruktor zapisuje statystyki do wskaźników i zwalnia pamięć. Funkcja update() obsługuje zdarzenia okna SFML, aktualizuje ruch postaci i czas symulacji, a także umożliwia zmianę mnożnika prędkości gry poprzez kliknięcia myszy. Metoda render() rysuje wszystkie elementy sceny w odpowiedniej kolejności, sortując obiekty według współrzędnej Y, aby poprawnie odwzorować głębię. Funkcja isValidPositions() sprawdza, czy dana pozycja na planszy jest dostępna dla postaci, tworząc w praktyce mapę ruchu. Plik spaja logikę gry i jej wizualizację, zapewniając pełną symulację działania restauracji wraz z interakcjami gracza i liczeniem statystyk.

3.6.4. Plik CharactersManager.cpp

Plik CharactersManager.cpp implementuje klasę CharactersManager, która zarządza wszystkimi postaciami w symulacji restauracji: kucharzami, kelnerami i klientami. Odpowiada za tworzenie postaci, ich ruch, przydzielanie zadań, zmianę stanów klientów i pracowników w zależności od sytuacji w restauracji, a także za aktualizację i renderowanie postaci w czasie rzeczywistym oraz zbieranie statystyk symulacji.

Tabela 3.8. Plik CharactersManager.cpp

```

#include "characters/CharactersManager.hpp"
#include "characters/Chef.hpp"
#include "characters/Waiter.hpp"
#include "characters/Customer.hpp"
#include "interiors/TablesManager.hpp"

#include <algorithm>
#include <random>
#include <unordered_map>

CharactersManager::CharactersManager(int scaleFactor, float tileWidth,
float tileHeight, int chefsNumber,
                                int waitersNumber, DishesManager*
dishesManager
                                ) : timeToAddCustomerDist(1.0f /
22.0f),
                                timeToRemoveCustomerDist(30.0f,
6.0f) {
    moveXSpeed = tileWidth;
    moveYSpeed = tileHeight;

    extern std::mt19937 globalRNG;
    timeToRemoveWaitingCustomer =
std::clamp(timeToRemoveCustomerDist(globalRNG), 15.0f, 50.0f);

```



```

        addChefs(scaleFactor, tileWidth,
                  moveXSpeed, chefsNumber,
                  dishesManager);
        addWaiters(scaleFactor, moveXSpeed, moveYSpeed, waitersNumber,
                   dishesManager);
    }

    CharactersManager::~CharactersManager() {
        for(Chef* chef : chefs) {
            delete chef;
        }
        chefs.clear();

        for(Waiter* waiter : waiters) {
            delete waiter;
        }
        waiters.clear();

        for(Customer* customer : waitingCustomers) {
            delete customer;
        }
        waitingCustomers.clear();

        for(Customer* customer : insideCustomers) {
            delete customer;
        }
        insideCustomers.clear();

        for(Customer* customer : resigningCustomers) {
            delete customer;
        }
        resigningCustomers.clear();

        for(Customer* customer : leavingCustomers) {
            delete customer;
        }
        leavingCustomers.clear();
    }

    void CharactersManager::addChefs(int scaleFactor, float tileWidth,
                                     float moveXSpeed, int chefsNumber,
                                     DishesManager* dishesManager) {
        for(int i = 0; i < chefsNumber; i++) {
            Chef* newChef = new Chef(scaleFactor, tileWidth, moveXSpeed,
                                     chefsStartPositions[i],
                                     chefsStartDirections[i], dishesManager);
            chefs.push_back(newChef);
        }
    }

    void CharactersManager::addWaiters(int scaleFactor, float moveXSpeed, float
    moveYSpeed,
                                     int waitersNumber,
                                     DishesManager* dishesManager) {
        for(int i = 0; i < waitersNumber; i++) {
            Waiter* newWaiter = new Waiter(scaleFactor, moveXSpeed, moveYSpeed,
                                     waitersStartPositions[i],
                                     waitersQueueHandlingPositions,
                                     waitersDishPickupPositions,
                                     waitersDishDropoffPositions,
    
```

```

        dishesManager);
    waiters.push_back(newWaiter);
}

void CharactersManager::addWaitingCustomer(int scaleFactor, float
tileWidth, float tileHeight,
float moveXSpeed, float
moveYSpeed) {
    totalCustomersNumberCounter++;
    extern std::mt19937 globalRNG;
    std::uniform_int_distribution<int> customerTextureDist{0, 1};
    Customer* newCustomer = new Customer(scaleFactor,
customersTexturesPaths[customerTextureDist(globalRNG)],
tileWidth, tileHeight, moveXSpeed,
moveYSpeed,
customersStartPositions,
customersQueueStartingPositions,
customersEnterRestaurantPositions,
customersExitRestaurantPositions);
    waitingCustomers.push_back(newCustomer);
    timeToAddCustomer = std::clamp(timeToAddCustomerDist(globalRNG), 8.0f,
40.0f);
}

void CharactersManager::moveWaitingCustomerToResignation() {
    extern std::mt19937 globalRNG;
    const int startIndex = 3;
    const int lastIndex = static_cast<int>(waitingCustomers.size()) - 1;
    const int range = lastIndex - startIndex;
    std::geometric_distribution<int> waitingCustomerIndexDist(0.35);
    int tailOffset = waitingCustomerIndexDist(globalRNG);
    if(tailOffset > range) {
        tailOffset = range;
    }
    int index = lastIndex - tailOffset;
    if(index >= 0 && index < static_cast<int>(waitingCustomers.size())) {
        Customer* customer = waitingCustomers[index];
        customer->changeToResigningState();
        resigningCustomers.push_back(customer);
        resigningCustomersNumberCounter++;
        waitingCustomers.erase(waitingCustomers.begin() + index);
    }
    timeToRemoveWaitingCustomer =
std::clamp(timeToRemoveCustomerDist(globalRNG), 15.0f, 50.0f);
}

void CharactersManager::moveWaitingCustomerToInside(int scaleFactor, int
tableNumber,
TablesManager*
tablesManager) {
    extern std::mt19937 globalRNG;
    std::bernoulli_distribution instantTableOccupationDist(0.25);
    bool occupyTableInstantly = instantTableOccupationDist(globalRNG);
    if(!waitingCustomers.empty()) {
        Customer* customer = waitingCustomers[0];
        if(customer->isWaitingToEnter()) {
            ChairPositionsAndDirections chairPositionsAndDirections =
tablesManager->getFreeChairPositions(tableNumber);
            if(occupyTableInstantly) {
                tablesManager->occupyTableInstantly(tableNumber);
            }
        }
    }
}

```

```

        }
        customer->setTableNumber(tableNumber);
        customer->setOccupyTableInstantly(occupyTableInstantly);
        customer->setIsLastCustomerOnTable(tablesManager-
>checkIsLastCustomerOnTable(tableNumber));
        customer-
>setTableNumberChairAndEnterChairPositions(scaleFactor,
tableNumber,
chairPositionsAndDirections.chairPositions,
chairPositionsAndDirections.enterChairPositions);
        customer-
>setChairHorizontalDirection(chairPositionsAndDirections.chairHorizontalDir
ection);
        customer-
>setChairVerticalDirection(chairPositionsAndDirections.chairVerticalDirecti
on);
        customer->changeToEnteredState();
        insideCustomers.push_back(customer);
        insideCustomersNumberCounter++;
        waitingCustomers.erase(waitingCustomers.begin());
    }
}

void CharactersManager::moveInsideCustomerToLeaving(int index,
TablesManager* tablesManager) {
    if(index >= 0 && index < static_cast<int>(insideCustomers.size())) {
        Customer* customer = insideCustomers[index];
        int tableNumber = customer->getTableNumber();
        Directions chairHorizontalDirection = customer-
>getChairHorizontalDirection();
        Directions chairVerticalDirection = customer-
>getChairVerticalDirection();
        tablesManager->freeChair(tableNumber, chairHorizontalDirection,
chairVerticalDirection);
        if(customer->getOccupyTableInstantly()) {
            tablesManager->freeInstantlyOccupiedTable(tableNumber);
            tablesManager->setWaitingToDishesTaken(tableNumber);
        }
        customer->changeToLeavingState();
        leavingCustomers.push_back(customer);
        leavingCustomersNumberCounter++;
        insideCustomers.erase(insideCustomers.begin() + index);
    }
}

void CharactersManager::removeResigningCustomer(int index) {
    if(index >= 0 && index < static_cast<int>(resigningCustomers.size())) {
        delete resigningCustomers[index];
        resigningCustomers.erase(resigningCustomers.begin() + index);
    }
}

void CharactersManager::removeLeavingCustomer(int index) {
    if(index >= 0 && index < static_cast<int>(leavingCustomers.size())) {
        delete leavingCustomers[index];
        leavingCustomers.erase(leavingCustomers.begin() + index);
    }
}

```

```

}

Waiter* CharactersManager::getNearestWaiterToPositions(Waiter*
nearestWaiter,
                                                    Waiter* newWaiter,
                                                    Positions positions)
{
    if(nearestWaiter == nullptr) {
        return newWaiter;
    }
    float actualNearestWaiterXPos = nearestWaiter->getXPos();
    float actualNearestWaiterYPos = nearestWaiter->getYPos();
    float distanceToActualNearestWaiter = std::sqrt(std::pow(positions.xPos
- actualNearestWaiterXPos, 2)
+
std::pow(positions.yPos - actualNearestWaiterYPos, 2));

    float newWaiterXPos = newWaiter->getXPos();
    float newWaiterYPos = newWaiter->getYPos();
    float distanceToNewWaiter = std::sqrt(std::pow(positions.xPos -
newWaiterXPos, 2)
+ std::pow(positions.yPos -
newWaiterYPos, 2));

    if(distanceToNewWaiter < distanceToActualNearestWaiter) {
        return newWaiter;
    }

    return nearestWaiter;
}

void CharactersManager::getNearestWaiterToQueueHandling(int scaleFactor,
Waiter* newWaiter) {
    Positions queueHandlingPositions{waitersQueueHandlingPositions.xPos *
scaleFactor,
                                                    waitersQueueHandlingPositions.yPos *
scaleFactor};
    queueNearestWaiter = getNearestWaiterToPositions(queueNearestWaiter,
newWaiter,
queueHandlingPositions);
}

bool CharactersManager::checkIsWaiterInQueueHandling() {
    for(Waiter* waiter : waiters) {
        if(waiter->getIsQueueHandling() ||
waiter->getIsSelectedToQueueHandling()) {
            return true;
        }
    }
    return false;
}

void CharactersManager::assignNearestWaiterToQueueHandling() {
    if(queueNearestWaiter != nullptr) {
        queueNearestWaiter->changeToQueueHandlingState();
    }
}

void CharactersManager::assignWaitersToTablesHandling(int scaleFactor,
TablesManager* tablesManager) {

```

```

        std::vector<int> waitingToHandlingTablesNumbers = tablesManager-
>getWaitingToHandlingTablesNumbers();
        for(int tableNumber : waitingToHandlingTablesNumbers) {
            Positions tableHandlingPositions = tablesManager-
>getTableHandlingPositions(tableNumber);
            Waiter* tableNearestWaiter = nullptr;
            for(Waiter* waiter : waiters) {
                if(!waiter->getIsAssignedToTask()) {
                    tableNearestWaiter =
getNearestWaiterToPositions(tableNearestWaiter,
                                                                    waiter,
tableHandlingPositions);
                }
            }
            if(tableNearestWaiter != nullptr) {
                Directions tableHandlingDirection = tablesManager-
>getTableHandlingDirection(tableNumber);
                tableNearestWaiter-
>setTableNumberPositionsAndDirection(scaleFactor, tableNumber,
tableHandlingPositions,
tableHandlingDirection);
                tableNearestWaiter->changeToTableHandlingState();
                tableNearestWaiter->setAssignedToTask(true);
                tablesManager->resetWaitingToHandling(tableNumber);
            }
        }
}

void CharactersManager::assignChefsToCooking(int tableNumber) {
    if(chefs.empty()) {
        return;
    }

    Chef* chefWithMinDishesNumber = chefs[0];
    int minDishesToCookForNumber = chefWithMinDishesNumber-
>getDishesToCookForNumber();

    for(Chef* chef : chefs) {
        int dishesToCookForNumber = chef->getDishesToCookForNumber();
        if(dishesToCookForNumber < minDishesToCookForNumber) {
            chefWithMinDishesNumber = chef;
            minDishesToCookForNumber = dishesToCookForNumber;
        }
    }

    chefWithMinDishesNumber->addTableToCookFor(tableNumber);
}

void CharactersManager::assignWaitersToDishPickup(int scaleFactor,
DishesManager*
dishesManager,
TablesManager*
tablesManager) {
    int readyDishTableNumber = dishesManager->getReadyDishTableNumber();
    Positions tableHandlingPositions = tablesManager-
>getTableHandlingPositions(readyDishTableNumber);
    Positions dishPickupPositions = {waitersDishPickupPositions.xPos *
scaleFactor,

```

```

waitersDishPickupPositions.yPos *
scaleFactor});
    Waiter* tableNearestWaiter = nullptr;
    for(Waiter* waiter : waiters) {
        if(!waiter->getIsAssignedToTask()) {
            tableNearestWaiter =
getNearestWaiterToPositions(tableNearestWaiter,
                                                                    waiter,
dishPickupPositions);
        }
    }
    if(tableNearestWaiter != nullptr) {
        dishesManager->setReadyDishWaitingForWaiter(true);
        Directions tableHandlingDirection = tablesManager-
>getTableHandlingDirection(readyDishTableNumber);
        tableNearestWaiter-
>setTableNumberPositionsAndDirection(scaleFactor, readyDishTableNumber,
tableHandlingPositions,
tableHandlingDirection);
        tableNearestWaiter->changeToDishPickupState();
        tableNearestWaiter->setAssignedToTask(true);
    }
}

void CharactersManager::assignCustomersToEating(DishesManager*
dishesManager) {
    std::vector<int> dishesOnTablesNumbers = dishesManager-
>getDishesOnTablesNumbers();
    std::unordered_map<int, int> tableOccupancyCount;
    for(int tableNumber : dishesOnTablesNumbers) {
        tableOccupancyCount[tableNumber]++;
    }

    for(int tableNumber : dishesOnTablesNumbers) {
        extern std::mt19937 globalRNG;
        std::lognormal_distribution<float> eatingTimeDist(2.5f, 0.35f);
        float eatingTime = std::clamp(eatingTimeDist(globalRNG), 11.0f,
28.0f);

        int customersAtTable = 0;
        for (Customer* customer : insideCustomers) {
            if(customer->getTableNumber() == tableNumber && customer-
>isSitting()) {
                customersAtTable++;
            }
        }

        if(customersAtTable == tableOccupancyCount[tableNumber]) {
            for(Customer* customer : insideCustomers) {
                if(customer->getTableNumber() == tableNumber && customer-
>isSitting()) {
                    customer->setEatingTime(eatingTime);
                    customer->changeToEatingState();
                }
            }
        }
    }
}

```

```

void CharactersManager::assignWaitersToDishDropoff(int scaleFactor,
                                                    TablesManager*
tablesManager) {
    std::vector<int> waitingToDishesTakenTablesNumbers = tablesManager-
>getWaitingToDishesTakenTablesNumbers();
    for(int tableNumber : waitingToDishesTakenTablesNumbers) {
        Positions tableHandlingPositions = tablesManager-
>getTableHandlingPositions(tableNumber);
        Waiter* tableNearestWaiter = nullptr;
        for(Waiter* waiter : waiters) {
            if(!waiter->getIsAssignedToTask()) {
                tableNearestWaiter =
getNearestWaiterToPositions(tableNearestWaiter,
                                                                    waiter,
tableHandlingPositions);
            }
            if(tableNearestWaiter != nullptr) {
                Directions tableHandlingDirection = tablesManager-
>getTableHandlingDirection(tableNumber);
                tableNearestWaiter-
>setTableNumberPositionsAndDirection(scaleFactor, tableNumber,
tableHandlingPositions,
tableHandlingDirection);
                tableNearestWaiter->changeToDishDropoffState();
                tableNearestWaiter->setAssignedToTask(true);
                tablesManager->resetWaitingToDishesTaken(tableNumber);
            }
        }
    }
}

void CharactersManager::update(float deltaTime, int scaleFactor,
                                float tileWidth, float tileHeight,
                                DishesManager* dishesManager,
                                TablesManager* tablesManager,
                                Pathfinder* pathFinder) {
    addCustomerTimer += deltaTime;
    if(addCustomerTimer > timeToAddCustomer && waitingCustomers.size() < 9)
    {
        addCustomerTimer = 0.0f;
        addWaitingCustomer(scaleFactor, tileWidth, tileHeight, moveXSpeed,
moveYSpeed);
    }
    if(waitingCustomers.size() > 5) {
        removeWaitingCustomerTimer += deltaTime;
        if(removeWaitingCustomerTimer > timeToRemoveWaitingCustomer) {
            removeWaitingCustomerTimer = 0.0f;
            moveWaitingCustomerToResignation();
        }
    }

    for(Chef* chef : chefs) {
        int addedDishesNumber = 0;
        chef->update(deltaTime, scaleFactor, &addedDishesNumber);
        preparedDishesNumberCounter += addedDishesNumber;
    }
}

```

```

        queueNearestWaiter = nullptr;
        bool isFreeTable = tablesManager->isFreeTable();
        bool isTablesWaitingToHandling = tablesManager-
>isTablesWaitingToHandling();
        bool isWaiterInQueueHandling = checkIsWaiterInQueueHandling();
        bool isReadyDishes = dishesManager->isReadyDishes();
        bool isReadyDishWaitingForWaiter = dishesManager-
>getReadyDishWaitingForWaiterStatus();
        bool isWaitingTablesToDishTaken = tablesManager-
>isTablesWaitingToDishesTaken();
        for(Waiter* waiter : waiters) {
            waiter->update(deltaTime, tileWidth, tileHeight, pathFinder);

            if(isFreeTable) {
                if(!waiter->getIsAssignedToTask() && !isWaiterInQueueHandling)
{
                    getNearestWaiterToQueueHandling(scaleFactor, waiter);
                }

                int tableNumber = tablesManager->getFreeTableNumber();
                if(waiter->getIsQueueHandling() && tablesManager-
>isFreeChair(tableNumber)) {
                    moveWaitingCustomerToInside(scaleFactor, tableNumber,
tablesManager);
                }
            }

            if(waiter->getIsNewOrder()) {
                int dishesNumber = tablesManager-
>getOccupiedChairsNumber(waiter->getTableNumber());
                for(int i = 0; i < dishesNumber; i++) {
                    assignChefsToCooking(waiter->getTableNumber());
                }
                waiter->setNewOrder(false);
                orderedDishesNumberCounter += dishesNumber;
            }

            if(waiter->getIsDishToPutdown()) {
                int tableNumber = waiter->getTableNumber();
                Positions dishesPositions = tablesManager-
>getDishesPositions(tableNumber);
                Positions scaleDishesPositions = {dishesPositions.xPos *
scaleFactor,
                                                    dishesPositions.yPos *
scaleFactor};
                dishesManager->moveMovingDishToDishesOnTables(tableNumber,
scaleDishesPositions);
                servedDishesNumberCounter++;
                waiter->setIsDishToPutdown(false);
            }
            if(waiter->getIsDishesToDropoff()) {
                int tableNumber = waiter->getTableNumber();
                int dishesOnTableNumber = 0;
                dishesManager->removeAllDishesOnTable(tableNumber,
&dishesOnTableNumber);
                droppedDishesNumberCounter += dishesOnTableNumber;
                waiter->setIsDishesToDropoff(false);
            }
        }
}

```



```

    if(isFreeTable && queueNearestWaiter != nullptr) {
        if(!queueNearestWaiter->getIsAssignedToTask() &&
!isWaiterInQueueHandling) {
            assignNearestWaiterToQueueHandling();
            queueNearestWaiter->setAssignedToTask(true);
        }
    }

    if(isTablesWaitingToHandling) {
        assignWaitersToTablesHandling(scaleFactor, tablesManager);
    }

    if(isReadyDishes && !isReadyDishWaitingForWaiter) {
        assignWaitersToDishPickup(scaleFactor, dishesManager,
tablesManager);
    }

    if(isWaitingTablesToDishTaken) {
        assignWaitersToDishDropoff(scaleFactor, tablesManager);
    }

    for(int i = 0; i < static_cast<int>(waitingCustomers.size()); i++) {
        if(i == 0) {
            waitingCustomers[i]->updateIfWaiting(deltaTime,
customersQueueStartingPositions.xPos * scaleFactor);
        }
        else {
            waitingCustomers[i]->updateIfWaiting(deltaTime,
waitingCustomers[i - 1]->getXPos() + tileWidth);
        }
    }
    for(int i = resigningCustomers.size() - 1; i >= 0; i--) {
        resigningCustomers[i]->updateIfResigning(deltaTime);
        if(resigningCustomers[i]->getAssignedToRemoveStatus()) {
            removeResigningCustomer(i);
        }
    }
    for(int i = 0; i < static_cast<int>(insideCustomers.size()); i++) {
        insideCustomers[i]->updateIfEntered(deltaTime,
                                            tileWidth, tileHeight,
                                            pathFinder,
                                            &eatenDishesNumberCounter);
        if(insideCustomers[i]->isWaitingToSit()) {
            int tableNumber = insideCustomers[i]->getTableNumber();
            if(tablesManager->getTableOccupiedStatus(tableNumber) &&
insideCustomers[i]->getIsLastCustomerOnTable()) {
                tablesManager->setWaitingToHandling(tableNumber);
            }
            insideCustomers[i]->changeToSittingState();
        }
        if(insideCustomers[i]->getLeaveRestaurantStatus()) {
            moveInsideCustomerToLeaving(i, tablesManager);
        }
    }
    for(int i = leavingCustomers.size() - 1; i >= 0; i--) {
        leavingCustomers[i]->updateIfLeaving(deltaTime);
        if(leavingCustomers[i]->getAssignedToRemoveStatus()) {
            removeLeavingCustomer(i);
        }
    }
}

```

```

    bool isDishesOnTables = dishesManager->isDishesOnTables();
    if(isDishesOnTables) {
        assignCustomersToEating(dishesManager);
    }
}

void CharactersManager::renderChefs(sf::RenderWindow* window) {
    std::vector<Chef*> sortedChefs(chefs.begin(), chefs.end());
    std::stable_sort(sortedChefs.begin(), sortedChefs.end(),
        [](Chef* left, Chef* right) {
            return left->getSortY() < right->getSortY();
        });
    for(Chef* chef : sortedChefs) {
        chef->render(window);
    }
}

void CharactersManager::renderWaitersAndInsideCustomers(sf::RenderWindow*
window) {
    for(Waiter* waiter : waiters) {
        waiter->render(window);
    }
    for(Customer* insideCustomer : insideCustomers) {
        insideCustomer->render(window);
    }
}

void
CharactersManager::renderWaitingResigningInsideLeavingCustomers(sf::RenderW
indow* window) {
    for(Customer* waitingCustomer : waitingCustomers) {
        waitingCustomer->render(window);
    }
    for(Customer* resigningCustomer : resigningCustomers) {
        resigningCustomer->render(window);
    }
    for(Customer* leavingCustomer : leavingCustomers) {
        leavingCustomer->render(window);
    }
}

std::vector<Character*> CharactersManager::getWaitersAndInsideCustomers() {
    std::vector<Character*> characters;
    characters.reserve(waiters.size() + insideCustomers.size());
    for(Waiter* waiter : waiters) {
        characters.push_back(waiter);
    }
    for(Customer* insideCustomer : insideCustomers) {
        characters.push_back(insideCustomer);
    }
    return characters;
}

void CharactersManager::getStatistics(int* totalCustomersNumberPtr, int*
resigningCustomersNumberPtr,
                                     int* insideCustomersNumberPtr, int*
leavingCustomersNumberPtr,
                                     int* orderedDishesNumberPtr, int*
preparedDishesNumberPtr,
                                     int* servedDishesNumberPtr, int*
eatenDishesNumberPtr,

```

```

                                int* droppedDishesNumberPtr) {
    *totalCustomersNumberPtr = totalCustomersNumberCounter;
    *resigningCustomersNumberPtr = resigningCustomersNumberCounter;
    *insideCustomersNumberPtr = insideCustomersNumberCounter;
    *leavingCustomersNumberPtr = leavingCustomersNumberCounter;
    *orderedDishesNumberPtr = orderedDishesNumberCounter;
    *preparedDishesNumberPtr = preparedDishesNumberCounter;
    *servedDishesNumberPtr = servedDishesNumberCounter;
    *eatenDishesNumberPtr = eatenDishesNumberCounter;
    *droppedDishesNumberPtr = droppedDishesNumberCounter;
}

```

Klasa `CharactersManager` inicjalizuje postaci w konstruktorze, ustawiając prędkości ruchu, dodając kucharzy i kelnerów oraz definiując losowe czasy dodawania i usuwania klientów. Destruktor zwalnia pamięć po wszystkich dynamicznie alokowanych obiektach. Plik zawiera funkcje zarządzania cyklem życia klientów – dodawanie ich do kolejki, przenoszenie do stanu rezygnacji, wpuszczanie do środka restauracji, a następnie wysyłanie do wyjścia, przy jednoczesnym aktualizowaniu statystyk.

`CharactersManager` realizuje logikę przydzielania najbliższego kelnera do obsługi kolejek, stołów, odbioru i dostarczania potraw oraz przydzielania zadań kucharzom w oparciu o minimalną liczbę już przypisanych zadań. Mechanizmy te wykorzystują różne rozkłady losowe, co umożliwia realistyczne symulowanie czasów oczekiwania i zachowań postaci.

Metoda `update` odpowiada za pełną symulację w każdym kroku czasowym. Dodaje nowych klientów według losowego czasu, usuwa tych, którzy rezygnują, aktualizuje ruchy i stany wszystkich klientów, kucharzy i kelnerów, zarządza nowymi zamówieniami, odbiorem i dostarczaniem potraw oraz przypisuje klientów do jedzenia. Iteruje przez wszystkie grupy klientów (oczekujący, rezygnujący, wewnątrzni, opuszczający lokal), zmieniając ich stany i usuwając opuszczających restaurację.

Dodatkowo plik zawiera funkcje renderowania kucharzy, kelnerów i klientów w odpowiedniej kolejności (z uwzględnieniem osi Y), aby uzyskać poprawny efekt nakładania się postaci w widoku 2D. Funkcja `getStatistics` gromadzi wszystkie dane dotyczące liczby klientów i potraw w różnych stanach: zamówione, przygotowane, podane, zjedzone i zrzucone, co pozwala na analizę przebiegu symulacji w czasie rzeczywistym.

4. WYNIKI SYMULACJI

4.1. Badanie wpływu liczby pracowników na efektywność funkcjonowania restauracji

Przetestowano różne kombinacje liczby kucharzy i kelnerów. W celu lepszej miarodajności wyników, dla każdej z nich wykonano po 3 serie pomiarowe po 60 minut, z liczbą stolików dwuosobowych równą 8 i czteroosobowych równą 3. Wyliczono średnie arytmetyczne. Przedstawiają to tabele poniżej.

Tabela 4.1. Tabela wyników dla chefs = 1, waiters = 1

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	167	171	181	173
Resigning customers	91	92	101	94.67
Inside customers	69	75	74	72.67
Leaving customers	56	60	65	60.33
Ordered dishes	69	73	73	71.67
Prepared dishes	57	63	66	62
Served dishes	57	61	65	61
Eaten dishes	56	60	65	60.33
Dropped dishes	56	53	57	55.33

Tabela 4.2. Tabela wyników dla chefs = 1, waiters = 2

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	180	168	172	173.33
Resigning customers	82	66	78	75.33
Inside customers	93	97	86	92
Leaving customers	72	72	67	70.33
Ordered dishes	88	95	84	89
Prepared dishes	69	72	65	68.67
Served dishes	69	71	64	68
Eaten dishes	72	72	67	70.33
Dropped dishes	69	71	62	67.33

Tabela 4.3. Tabela wyników dla chefs = 1, waiters = 3

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	168	180	183	177
Resigning customers	71	78	83	77.33
Inside customers	91	95	94	93.33
Leaving customers	69	73	72	71.33
Ordered dishes	91	95	90	92
Prepared dishes	69	70	71	70
Served dishes	69	69	71	69.67
Eaten dishes	69	75	72	72
Dropped dishes	67	68	70	68.33

Tabela 4.4. Tabela wyników dla chefs = 2, waiters = 1

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	174	173	178	175.00
Resigning customers	95	93	93	93.67
Inside customers	73	75	78	75.33
Leaving customers	68	72	72	70.67
Ordered dishes	73	73	74	73.33
Prepared dishes	71	72	74	72.33
Served dishes	70	72	72	71.33
Eaten dishes	68	72	72	70.67
Dropped dishes	53	53	56	54

Tabela 4.5. Tabela wyników dla chefs = 2, waiters = 2

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	178	190	163	177
Resigning customers	39	41	16	32
Inside customers	138	145	141	141.33
Leaving customers	119	123	124	122
Ordered dishes	131	141	141	137.67
Prepared dishes	121	131	126	126
Served dishes	119	128	124	123.67
Eaten dishes	119	123	124	122
Dropped dishes	116	122	122	120

Tabela 4.6. Tabela wyników dla chefs = 2, waiters = 3

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	192	179	186	185.67
Resigning customers	36	24	19	26.33
Inside customers	147	150	163	153.33
Leaving customers	128	125	139	130.67
Ordered dishes	145	149	160	151.33
Prepared dishes	131	127	136	131.33
Served dishes	130	127	135	130.67
Eaten dishes	128	127	139	131.33
Dropped dishes	127	124	133	128

Tabela 4.7. Tabela wyników dla chefs = 3, waiters = 1

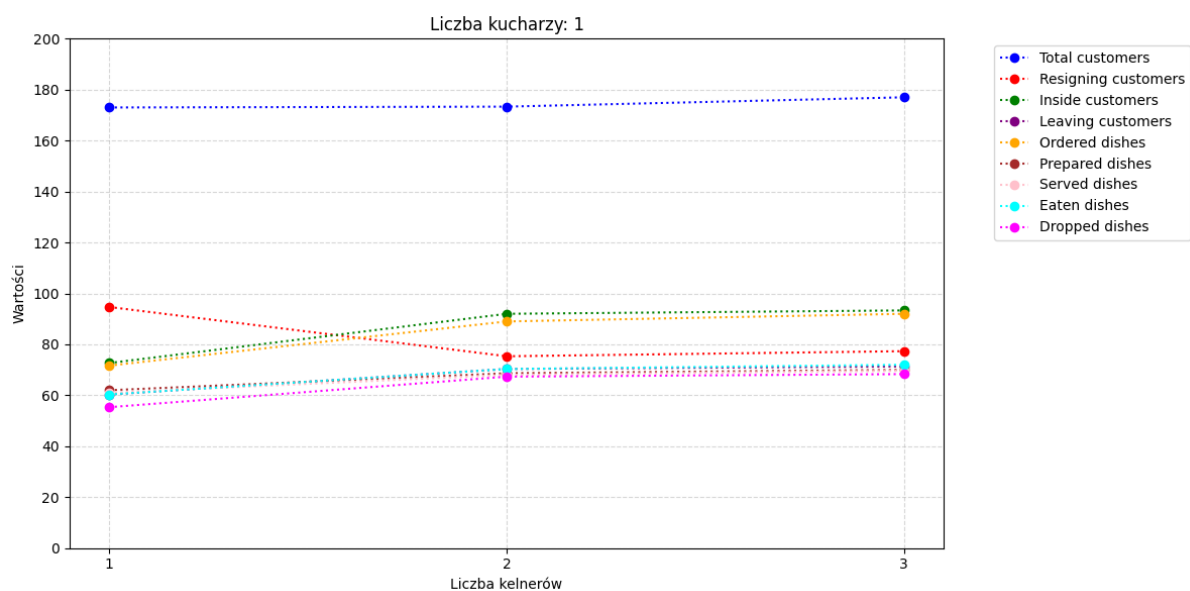
	Próba 1	Próba 2	Próba 3	Średnia
Total customers	172	166	182	173.33
Resigning customers	92	85	100	92.33
Inside customers	74	74	74	74
Leaving customers	70	68	71	69.67
Ordered dishes	74	74	72	73.33
Prepared dishes	70	72	71	71
Served dishes	70	69	71	70
Eaten dishes	70	68	71	69.67
Dropped dishes	50	48	56	51.33

Tabela 4.8. Tabela wyników dla chefs = 3, waiters = 2

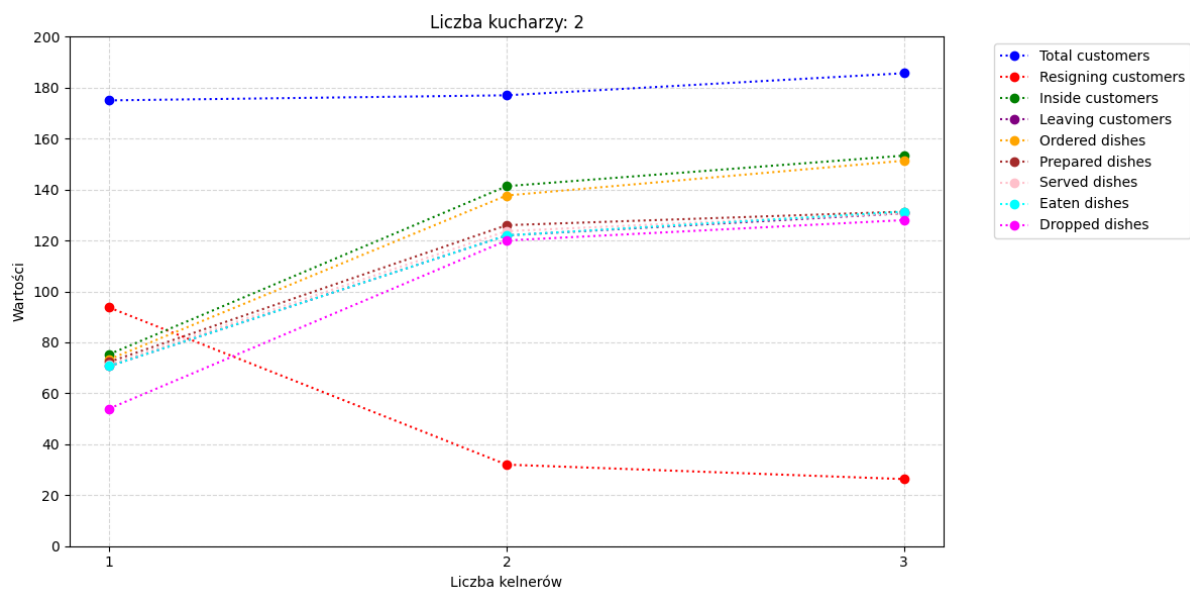
	Próba 1	Próba 2	Próba 3	Średnia
Total customers	191	172	176	179.67
Resigning customers	36	21	27	28
Inside customers	148	146	145	146.33
Leaving customers	133	133	138	134.67
Ordered dishes	144	145	144	144.33
Prepared dishes	144	143	139	142
Served dishes	135	134	139	136
Eaten dishes	133	133	138	134.67
Dropped dishes	129	128	121	126

Tabela 4.9. Tabela wyników dla chefs = 3, waiters = 3

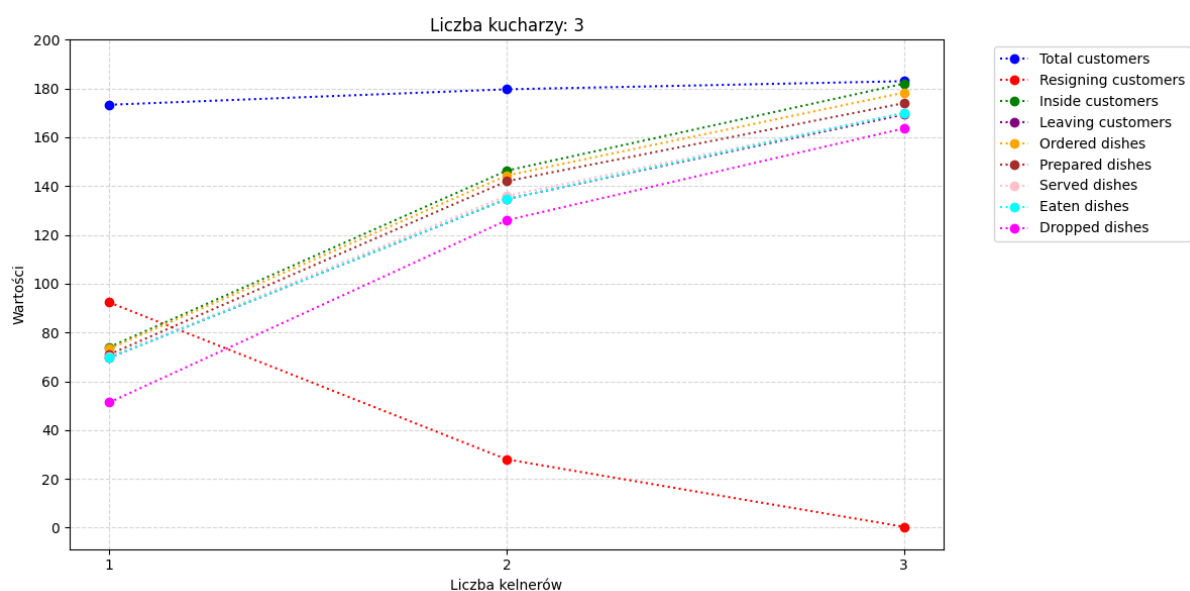
	Próba 1	Próba 2	Próba 3	Średnia
Total customers	171	193	185	183,00
Resigning customers	0	1	0	0.33
Inside customers	169	192	185	182
Leaving customers	156	180	172	169.33
Ordered dishes	166	188	181	178.33
Prepared dishes	161	184	177	174
Served dishes	158	179	173	170
Eaten dishes	158	180	172	170
Dropped dishes	152	171	168	163.67



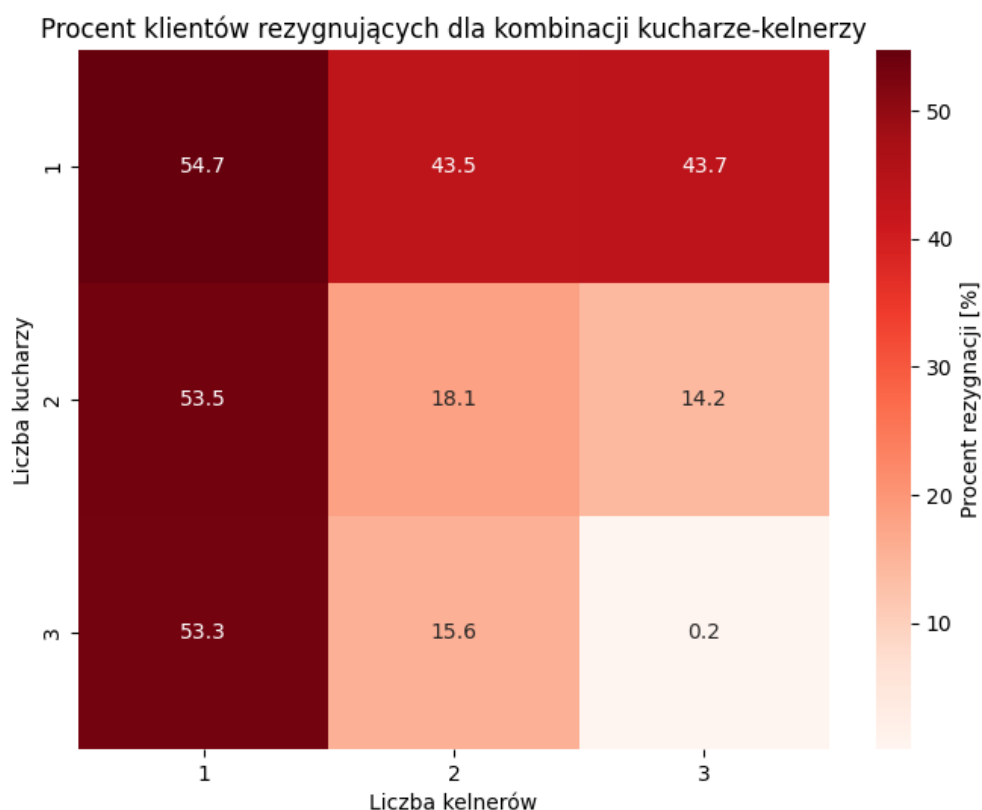
Rysunek 4.1. Wykres przedstawiający wyniki dla liczby kucharzy równej 1 i każdej możliwej liczby kelnerów



Rysunek 4.2. Wykres przedstawiający wyniki dla liczby kucharzy równej 2 i każdej możliwej liczby kelnerów



Rysunek 4.3. Wykres przedstawiający wyniki dla liczby kucharzy równej 3 i każdej możliwej liczby kelnerów



Rysunek 4.4. Mapa ciepła przedstawiająca procent rezygnujących klientów dla kombinacji kucharze-kelnerzy

Analiza przeprowadzonych symulacji pozwala na sformułowanie kluczowych wniosków dotyczących optymalizacji zasobów ludzkich w badanym modelu restauracji:

1. Wąskie gardło w obsłudze kelnerskiej

Zestawienie danych jednoznacznie wskazuje, że przy niskiej liczbie kelnerów (szczególnie przy 1 kelnerze), system generuje największe straty. Nawet zwiększenie liczby kucharzy do 3 nie rozwiązuje problemu, jeśli obsługa stolików jest niewydolna – w takim scenariuszu procent rezygnacji klientów utrzymuje się na drastycznie wysokim poziomie (powyżej 50%).

2. Korelacja między liczbą kelnerów a rezygnacjami

Największy spadek liczby klientów rezygnujących (Resigning customers) obserwujemy przy przejściu z 1 na 2 kelnerów. Jak pokazuje mapa ciepła, dla 3 kucharzy zwiększenie obsady kelnerskiej z 1 do 3 osób pozwoliło zredukować współczynnik rezygnacji z 53,3% do niemal zerowego poziomu 0,2%.

3. Efektywność zbierania naczyń

Liczba zebranych brudnych naczyń jest bezpośrednio powiązana z przepustowością personelu obsługującego salę. Przy 1 kelnerze, niezależnie od liczby kucharzy, wartość ta jest najniższa (ok. 50-60), co blokuje rotację stolików. Zwiększenie liczby kelnerów do 3 drastycznie podnosi ten wskaźnik (do poziomu 163-170), co pozwala na płynną obsługę większej liczby gości (Inside customers).

4. Priorytety zatrudnienia z perspektywy pracodawcy

Z punktu widzenia optymalizacji kosztów zatrudnienia, bardziej efektywnym rozwiązaniem jest dołożenie etatu kelnerskiego niż kucharskiego. Analiza mapy ciepła wykazuje, że zwiększenie liczby kucharzy przy stałej, małej liczbie kelnerów daje znikome rezultaty w redukcji rezygnacji (zmiana z 54,7% na 53,3% przy 1 kelnerze). Natomiast dodanie jednego kelnera przy stałej liczbie kucharzy pozwala na skokową poprawę wydajności całego systemu. Kelner pełni rolę "wentyla" bezpieczeństwa – bez sprawnej obsługi sali i sprawnego sprzątania naczyń moce przerobowe kuchni pozostają niewykorzystane.

5. Punkt optymalny

Najwyższą efektywność operacyjną (najwięcej zjedzonych dań, najmniej rezygnacji) osiągnięto w konfiguracji 3 kucharzy i 3 kelnerów. Jednakże konfiguracja 2 kucharzy i 3 kelnerów (14,2% rezygnacji) również wypada znacznie lepiej niż jakakolwiek konfiguracja z tylko jednym kucharzem lub kelnerem, co potwierdza tezę o kluczowej roli personelu sali w procesie generowania przychodu.

4.2. Badanie wpływu liczby stolików na efektywność funkcjonowania restauracji

Przetestowano różne kombinacje liczby stolików dwuosobowych i czteroosobowych. W celu lepszej miarodajności wyników, dla każdej z nich wykonano po 3 serie pomiarowe po 60 minut, z liczbą kucharzy równą 2 i kelnerów również równą 2. Wyliczono średnie arytmetyczne. Przedstawiają to tabele poniżej.

Tabela 4.10. Tabela wyników dla twoChairsTables = 8, fourChairsTables = 0

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	186	176	187	183
Resigning customers	47	37	41	41.67
Inside customers	136	135	139	136.67
Leaving customers	123	123	128	124.67
Ordered dishes	133	133	137	134.33
Prepared dishes	124	125	130	126.33
Served dishes	123	125	128	125.33
Eaten dishes	123	123	128	124.67
Dropped dishes	121	122	124	122.33

Tabela 4.11. Tabela wyników dla twoChairsTables = 0, fourChairsTables = 3

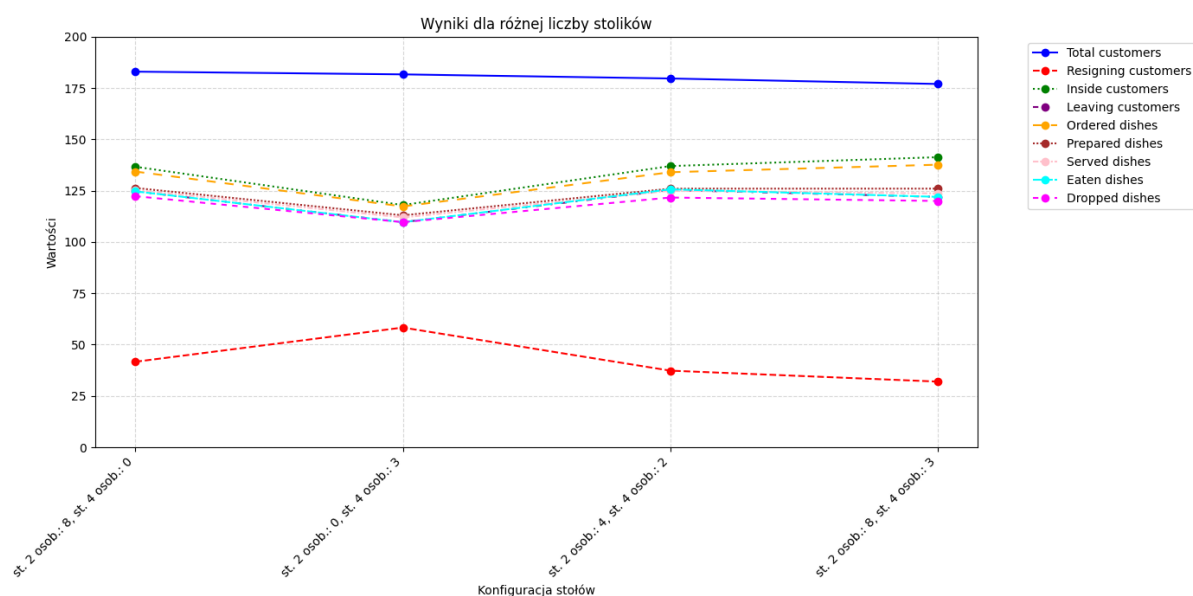
	Próba 1	Próba 2	Próba 3	Średnia
Total customers	181	175	189	181.67
Resigning customers	59	50	66	58.33
Inside customers	115	121	118	118
Leaving customers	110	109	110	109.67
Ordered dishes	114	121	117	117.33
Prepared dishes	112	114	113	113
Served dishes	112	113	111	112
Eaten dishes	110	109	110	109.67
Dropped dishes	110	109	110	109.67

Tabela 4.13. Tabela wyników dla twoChairsTables = 4, fourChairsTables = 2

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	172	177	190	179.67
Resigning customers	33	31	48	37.33
Inside customers	134	144	133	137
Leaving customers	123	132	121	125.33
Ordered dishes	132	138	132	134
Prepared dishes	125	130	123	126
Served dishes	124	129	122	125
Eaten dishes	123	132	122	125.67
Dropped dishes	121	126	118	121.67

Tabela 4.14. Tabela wyników dla twoChairsTables = 8, fourChairsTables = 3

	Próba 1	Próba 2	Próba 3	Średnia
Total customers	178	190	163	177
Resigning customers	39	41	16	32
Inside customers	138	145	141	141.33
Leaving customers	119	123	124	122
Ordered dishes	131	141	141	137.67
Prepared dishes	121	131	126	126
Served dishes	119	128	124	123.67
Eaten dishes	119	123	124	122
Dropped dishes	116	122	122	120



Rysunek 4.5. Wykres przedstawiający wyniki dla wszystkich przetestowanych kombinacji liczby stolików

Przeprowadzona analiza wpływu konfiguracji sali (liczby stolików dwu- i czteroosobowych) na wskaźniki operacyjne restauracji pozwala na wyciągnięcie następujących wniosków.

1. Kluczowe znaczenie stolików dwuosobowych

Najmniej korzystną konfiguracją okazał się wariant posiadający jedynie 3 stoliki czteroosobowe przy całkowitym braku stolików dwuosobowych. Odnotowano w nim najwyższą średnią liczbę klientów rezygnujących (58,33) oraz najniższą liczbę zjedzonych dań (109,67). Świadczy to o tym, że mała liczba punktów obsługi (nawet o większej pojemności) drastycznie ogranicza przepustowość systemu przy rozproszonym strumieniu klientów.

2. Optymalizacja współczynnika rezygnacji

Najniższą liczbę rezygnacji (32 osoby) uzyskano w konfiguracji mieszanej o największej łącznej liczbie miejsc (8 stolików 2-osobowych i 3 stoliki 4-osobowe). Zwiększenie całkowitej liczby dostępnych stolików pozwala na płynniejsze przyjmowanie grup o różnej wielkości, co bezpośrednio przekłada się na mniejszą stratę potencjalnych klientów już na wejściu.

3. Stabilność wydajności kuchni i serwisu

Wskaźniki takie jak Prepared dishes oraz Served dishes wykazują dużą korelację z liczbą klientów wewnątrz restauracji (Inside customers). W konfiguracjach posiadających co najmniej 8 stolików dwuosobowych (lub ich ekwiwalent w miksie 4/2), liczba serwowanych dań stabilizuje się na poziomie powyżej 120-125, co sugeruje, że przy 2 kucharzach i 2 kelnerach jest to blisko górnej granicy wydajności personelu przy danej infrastrukturze.

4. Relacja między pojemnością sali a sprzątaniem naczyń

Wartość Dropped dishes (sprzątnięte brudne naczynia) jest najwyższa w konfiguracjach o większej liczbie stolików dwuosobowych. Wynika to z faktu, że większa liczba mniejszych stolików pozwala na szybszą rotację grup klientów – mniejsze stoliki są częściej zwalniane i wymagają częstszego sprzątania, co napędza obieg naczyń w systemie.

5. Wnioski dla zarządcy sali restauracyjnej

- Najlepsze wyniki operacyjne (wysoka liczba Inside customers i niskie rezygnacje) osiąga się, łącząc dużą liczbę stolików 2-osobowych z kilkoma 4-osobowymi.
- Model pokazuje, że lepiej mieć więcej punktów obsługi (więcej stolików), niż mniej stolików o dużej liczbie miejsc. Konfiguracja 8x2 os. wypada znacznie lepiej niż 3x4 os., mimo że łączna liczba miejsc jest zbliżona (16 vs 12), ponieważ pozwala na obsługę większej liczby niezależnych grup klientów jednocześnie.

5. PODSUMOWANIE

Celem niniejszego projektu było zaprojektowanie i zaimplementowanie interaktywnej aplikacji do symulacji pracy restauracji w języku C++ z wykorzystaniem biblioteki graficznej SFML. Projekt łączy w sobie aspekty modelowania systemów dynamicznych, programowania obiektowego oraz wizualizacji danych w czasie rzeczywistym. Opracowano zaawansowany model funkcjonowania lokalu, obejmujący interakcje między klientami, kelnerami i kucharzami, uwzględniając procesy od przyjmowania gości, przez przygotowywanie posiłków, aż po sprzątanie stanowisk. Aby nadać symulacji cechy rzeczywistego systemu, zastosowano rozkłady prawdopodobieństwa, takie jak rozkład wykładniczy dla napływu gości, rozkład log-normalny dla czasu jedzenia oraz rozkład gamma dla czasu gotowania.

Analiza wyników przeprowadzonych symulacji pozwala na sformułowanie kluczowych wniosków dotyczących optymalizacji zasobów. Badania jednoznacznie wskazują na istnienie „wąskiego gardła” w obsłudze kelnerskiej. Przy obsadzie składającej się z tylko jednego kelnera system generuje największe straty, a współczynnik rezygnacji klientów utrzymuje się na poziomie powyżej 50%, nawet przy zwiększeniu liczby kucharzy. Największą poprawę wydajności zaobserwowano przy zwiększeniu obsady kelnerskiej z jednej do dwóch osób. W scenariuszu z trzema kucharzami, przejście z jednego do trzech kelnerów pozwoliło zredukować rezygnacje z 53,3% do niemal zerowego poziomu 0,2%. Wynika to z faktu, że liczba zebranych brudnych naczyń bezpośrednio warunkuje rotację stolików – bez sprawnej obsługi sali moce przerobowe kuchni pozostają niewykorzystane. Z perspektywy zarządczej bardziej efektywnym rozwiązaniem jest inwestycja w personel kelnerski, który pełni rolę „wentyla bezpieczeństwa” całego systemu. Punkt optymalny wydajności operacyjnej osiągnięto w konfiguracji trzech kucharzy i trzech kelnerów.

Równie istotne wnioski płyną z analizy konfiguracji sali i rodzaju stolików. Badanie wykazało, że kluczowe znaczenie dla przepustowości mają stoliki dwuosobowe. Najmniej korzystnym wariantem okazała się konfiguracja oparta wyłącznie na stolikach czteroosobowych, co skutkowało najwyższą liczbą rezygnacji i najniższą liczbą zjedzonych dań. Dowodzi to, że mała liczba punktów obsługi, nawet o większej pojemności, drastycznie ogranicza system przy rozproszonym strumieniu klientów. Najlepsze wyniki operacyjne uzyskano w strukturze mieszanej, łączącej 8 stolików dwuosobowych i 3 stoliki czteroosobowe. Większa liczba mniejszych stolików promuje szybszą rotację grup i pozwala na płynniejszą obsługę różnych kategorii gości.

Wnioski końcowe wskazują, że elastyczność infrastruktury jest ważniejsza niż surowa liczba miejsc – lepiej posiadać więcej niezależnych punktów obsługi niż kilka dużych stolików. Zrealizowana aplikacja stanowi skuteczne narzędzie do analizy wydajności i pozwala na wyciągnięcie praktycznych wniosków dotyczących optymalizacji zasobów bez konieczności ingerencji w rzeczywisty system. Dzięki modułowej architekturze opartej na maszynach stanów, system jest łatwy w rozbudowie o kolejne parametry analityczne.